



河南大學
HENAN UNIVERSITY

计算机操作系统

主讲人：胡 萍

明德，新民

止于至善

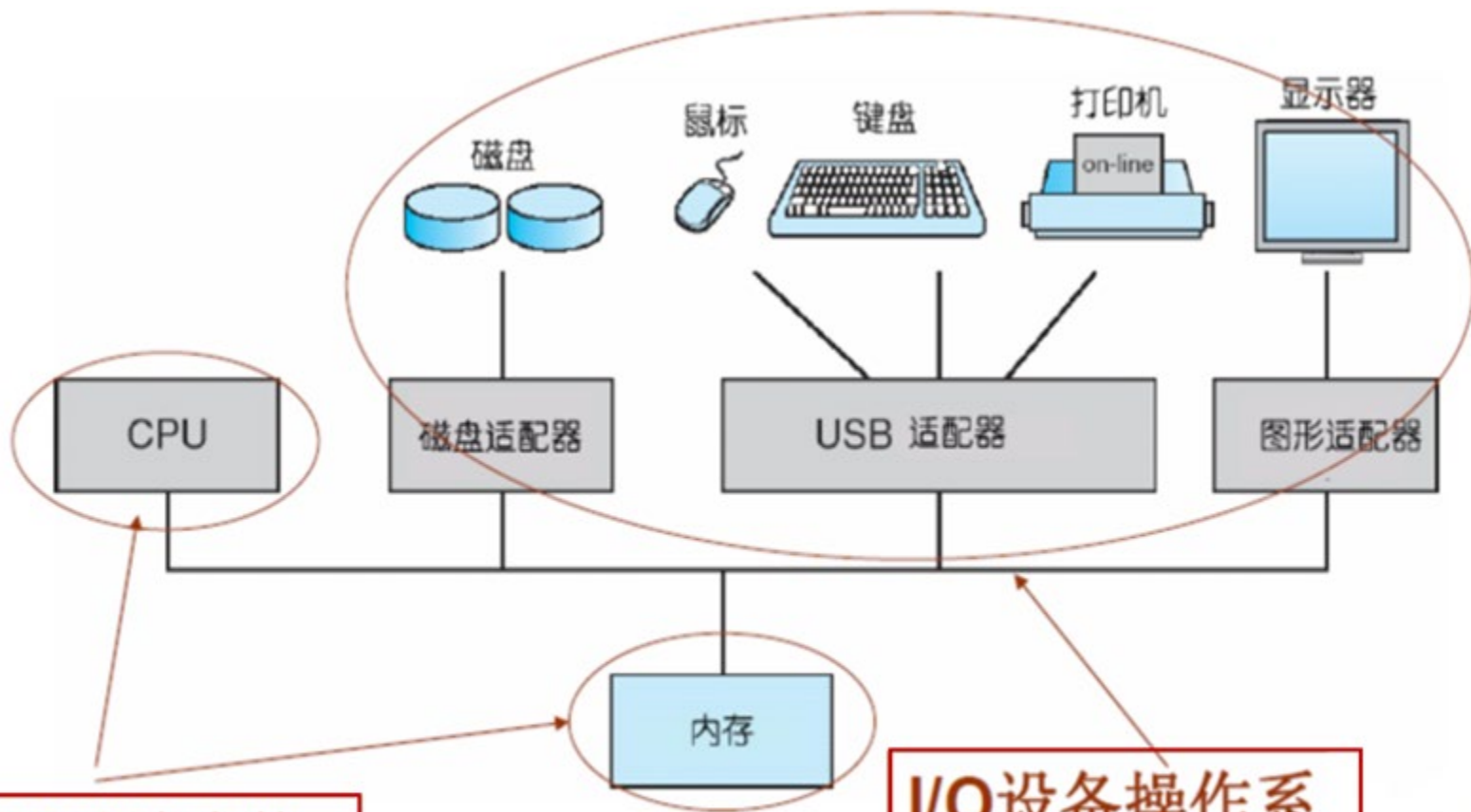
第六章 输入输出系统 (I/O系统)

第1讲





计算机体系结构



CPU和内存的管理都已学过

I/O设备操作系统怎样管理？

本章内容

6.1 I/O系统的功能、模型和接口

6.2 I/O设备和设备控制器

6.3 中断机构和中断处理程序

6.4 设备驱动程序

6.5 与设备无关的I/O软件

6.6 用户层的IO软件

6.7 缓冲区管理

6.8 磁盘存储器的性能和调度





6.1 I/O系统的功能、模型和接口

- 6.1.1 I/O系统的基本功能

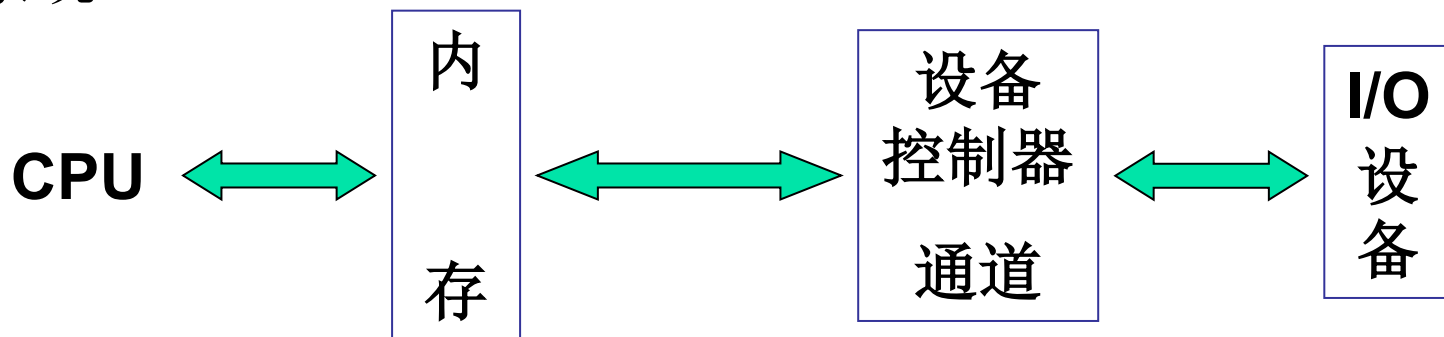
- 6.1.2 I/O系统的层次结构和模型

- 6.1.3 I/O系统接口



I/O系统简介

□ I/O系统：I/O系统是用于实现数据的输入、输出及数据存储的系统。



□ I/O系统管理的对象：I/O设备、设备控制器、I/O通道。

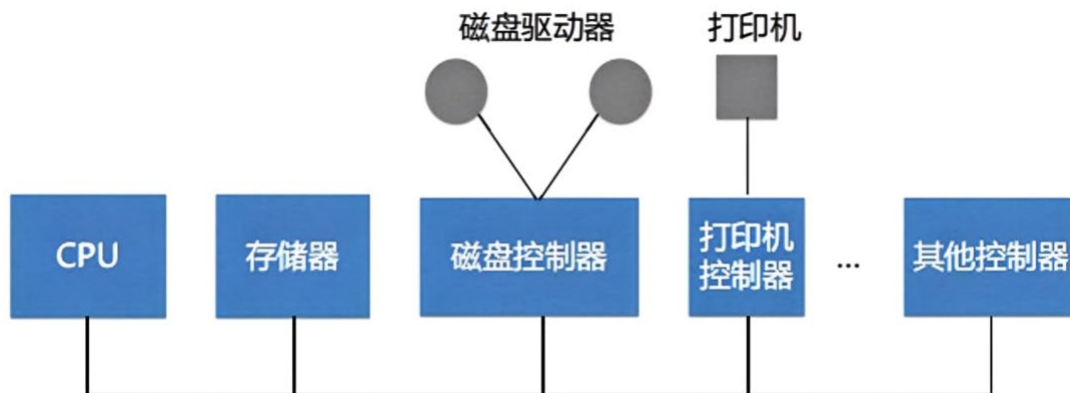
□ I/O系统最主要任务：完成用户提出的I/O请求，提高I/O速率，以及提高设备的利用率，并能为更高层的进程方便地使用这些设备提供手段。



I/O系统简介

□ 微机I/O系统

- CPU不直接与I/O通信，是通过**设备控制器**。
- **设备控制器**作为CPU与I/O的接口。



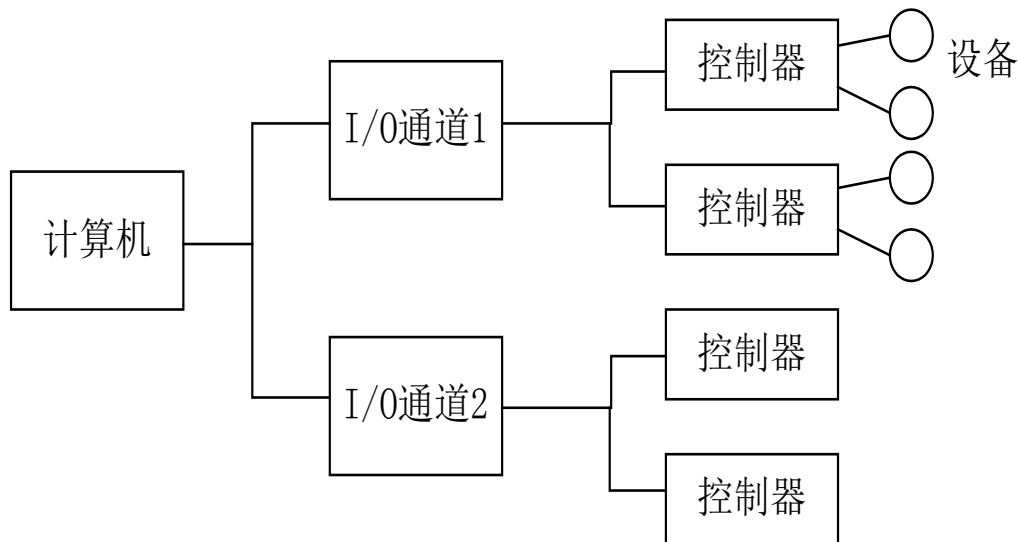


I/O系统简介

□ 主机I/O系统（通道I/O系统结构）

一般主机所配的I/O较多，为减轻CPU的负担，通常采用如下四级I/O结构：

- I/O设备
- 设备控制器
- **I/O通道**
- 计算机





6.1.1 I/O系统的基本功能

1. 隐藏物理设备的实现细节

通常为I/O设备配置**设备控制器**，用户通过命令和参数控制外部设备。

2. 与设备的无关性

与设备的无关性是在**隐藏物理设备细节**的基础上实现的。它允许通过**逻辑设备名**来使用某设备，也可以**实现I/O重定向**。如Windows系统，可以为I/O设备自动安装和更新驱动程序。

1、2为了方便用户



6.1.1 I/O系统的基本功能

3. 提高处理机和I/O设备的利用率 **3、4为了提高CPU和I/O利用率**

I/O系统要尽可能地让处理机和I/O设备并行操作，以提高它们的**利用率**。为此，一方面要求处理机能**快速响应用户的I/O请求**，使I/O设备尽快地运行起来；另一方面也应**尽量减少在每个I/O设备运行时处理机的干预时间**。

4. 对I/O设备进行控制

对I/O设备进行控制是驱动程序的功能。目前对I/O设备有**四种控制方式**：① 采用轮询的可编程I/O方式；② 采用中断的可编程I/O方式；③ 直接存储器访问方式；④ I/O通道方式。



6.1.1 I/O系统的基本功能

5. 确保对设备的正确共享

从设备的共享属性上，可将系统中的设备分为如下两类：

- **独占设备**：互斥地访问。如打印机、磁带机等
- **共享设备**：允许同时访问。如磁盘。

6. 错误处理

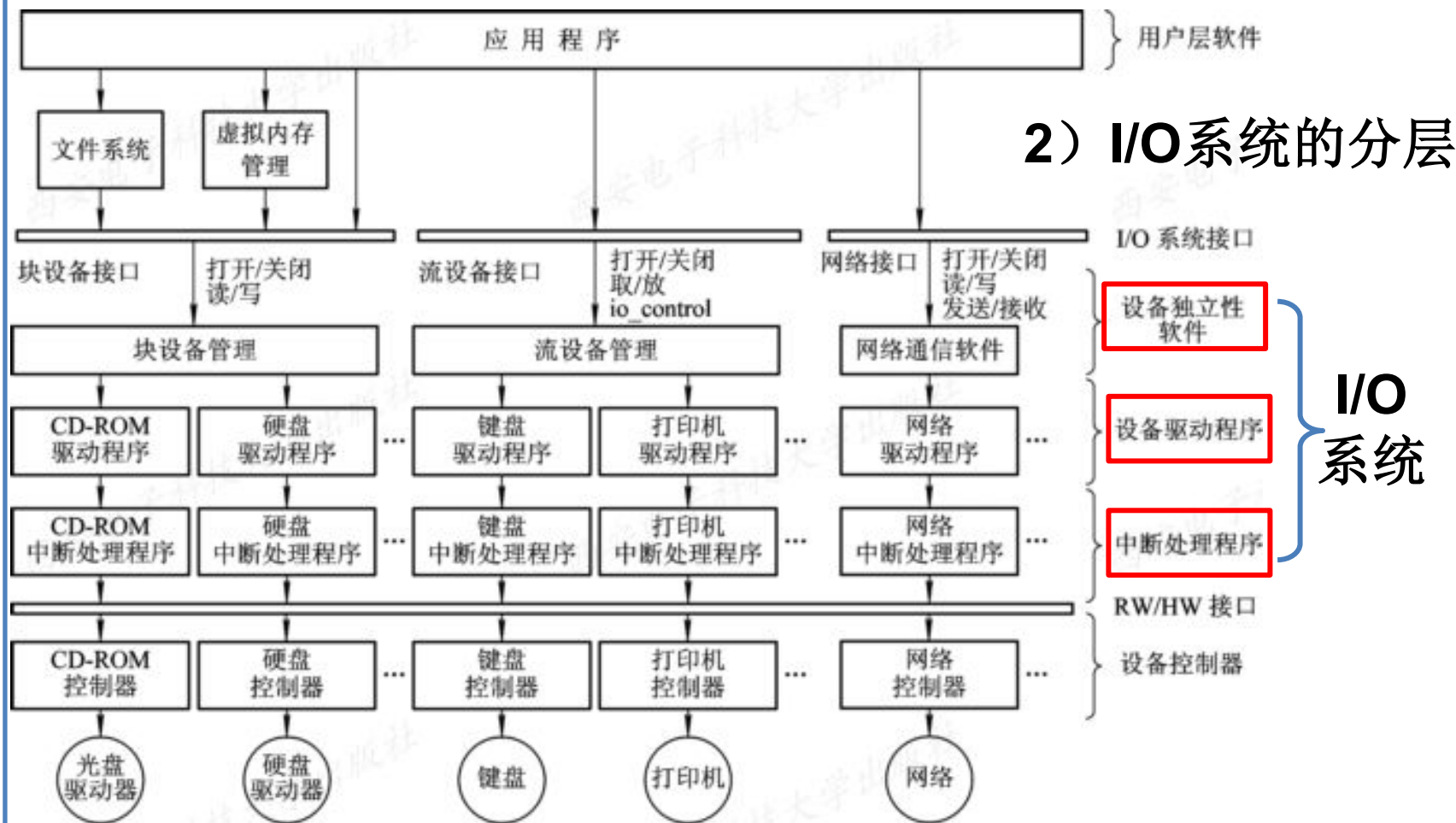
5、6为了方便设备共享

大多数的设备都包括较多的机械和电气部分，运行时容易出现错误和故障。从处理的角度，可将错误分为临时性错误和持久性错误。对于临时性错误，可通过重试操作来纠正，只有在发生了持久性错误时，才需要向上层报告。



6.1.2 I/O系统的层次结构和模型

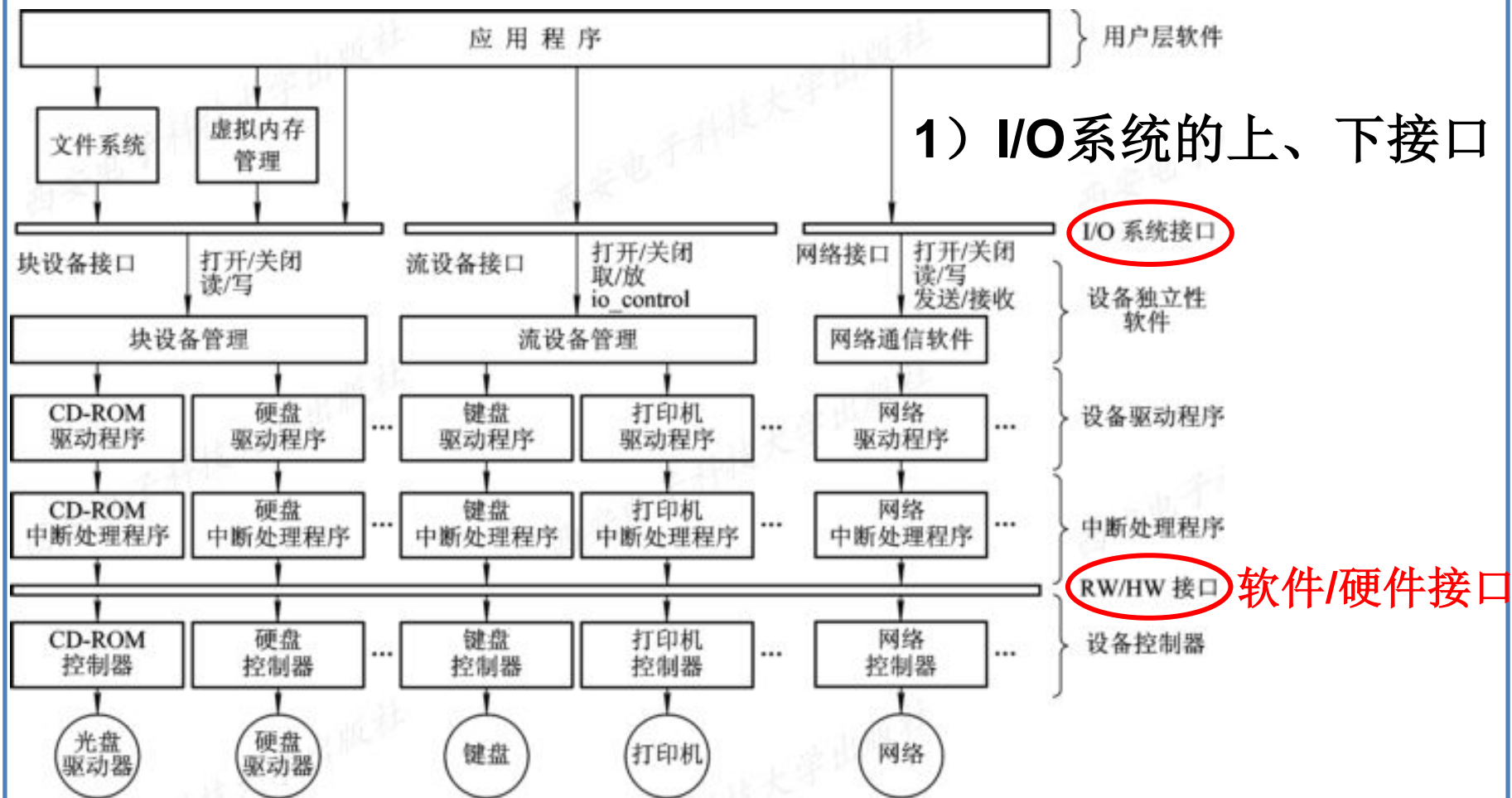
2. I/O系统中各种模块之间的层次视图





6.1.2 I/O系统的层次结构和模型

2. I/O系统中各种模块之间的层次视图

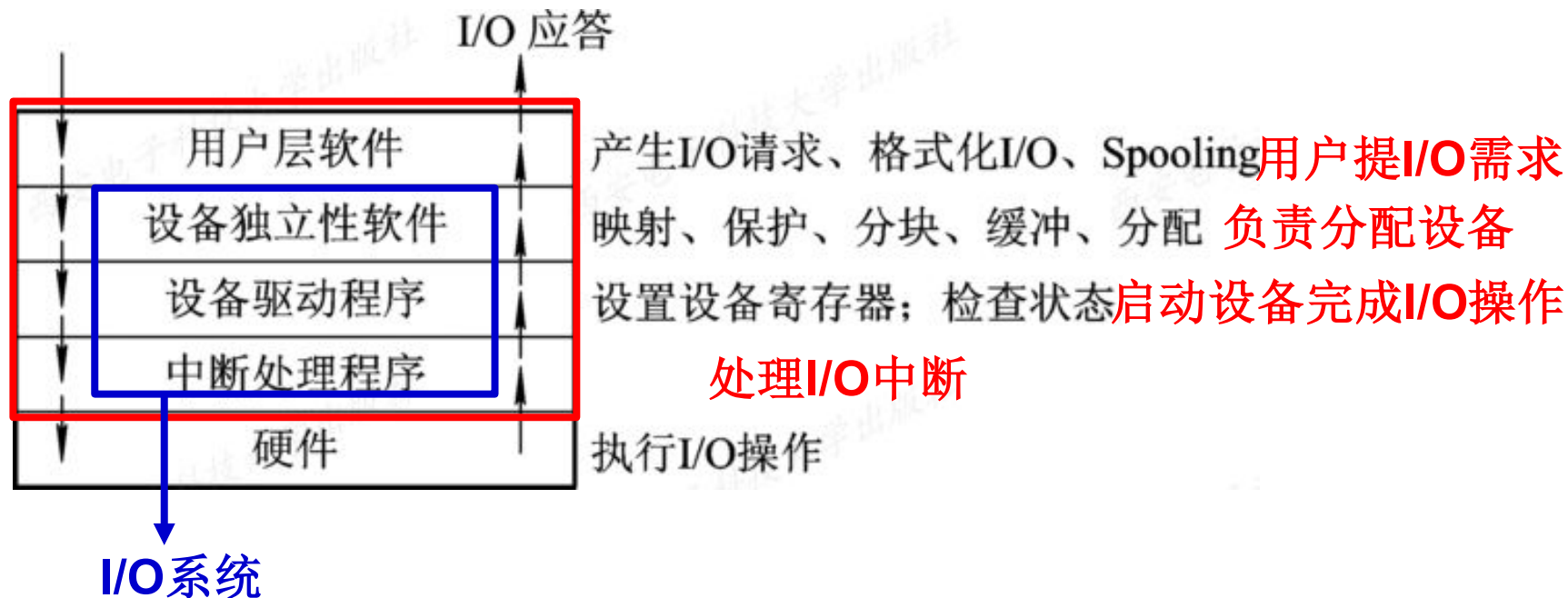




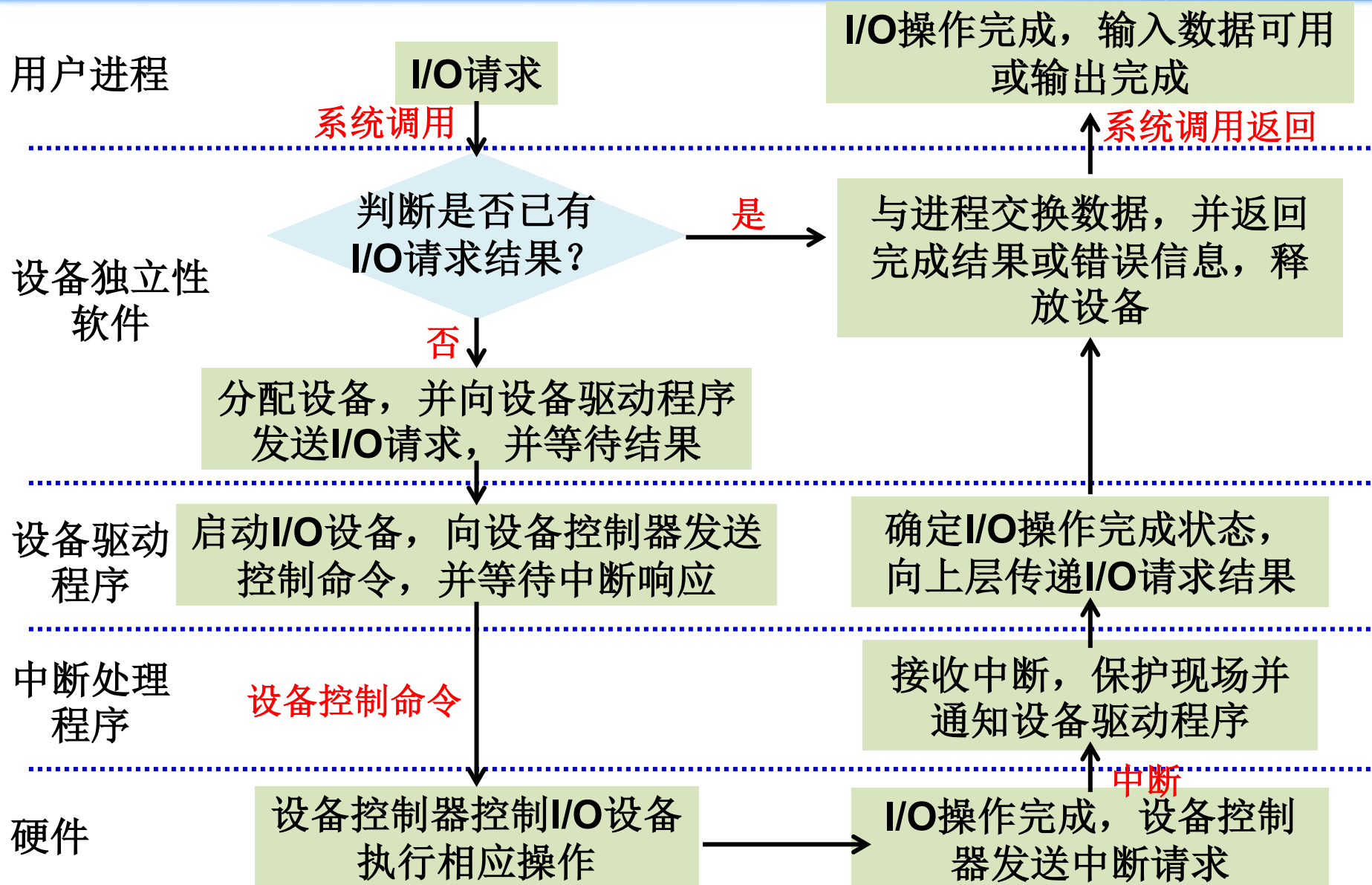
6.1.2 I/O系统的层次结构和模型

1. I/O软件的层次结构

通常把I/O 软件组织成四个层次，如图。



I/O请求的生存周期





6.1.3 I/O系统接口

1. 块设备接口

块设备接口是块设备管理程序与高层之间的接口，反映了大部分磁盘、光盘的本质特征，用于控制该类设备的输入与输出。

- **块设备**：数据的存取和传输以**数据块**为单位的设备。传输速率**高**、**可寻址**、**可随机读写**。常采用**DMA**方式，如磁盘。
- 隐藏了磁盘的三维结构。对所有扇区进行统一编号，将扇区三维地址（柱面号、盘面号和扇区号）转化为一维线性序列。
- 将抽象命令映射为底层操作。该接口将上层的抽象命令转换为较底层能识别的具体命令。如将一维的逻辑块号转化为三维地址（盘面号、磁道号、扇区号）。



6.1.3 I/O系统接口

2. 流设备接口（字符设备接口）

流设备接口是流设备管理程序与高层之间的接口。该接口又称为**字符设备接口**，它反映了大部分字符设备的本质特征，用于控制字符设备的输入或输出。

- **字符设备**：数据的存取和传输以**字符**为单位的设备。传输速率较低，**不可寻址**、采用顺序存取方式。输入输出时，常采用**中断驱动**的方式。如键盘、打印机。
- **get和put操作**：通常建立一个字符缓冲区，**get**用于从缓冲区中取得一个字符，**put**用于把一个字符输送到缓冲区去。
- **in-control指令**：用于统一管理各种类型的字符设备。



6.1.3 I/O系统接口

3. 网络通信接口

在现代OS中，都提供了面向网络的功能。但首先还需要通过某种方式把计算机连接到网络上。同时操作系统也必须提供相应的网络软件和网络通信接口，使计算机能通过网络与网络上的其它计算机进行通信或上网浏览。



6.2 I/O设备和设备控制器

□ 6.2.1 I/O设备

□ 6.2.2 设备控制器

□ 6.2.3 内存映像I/O

□ 6.2.4 I/O通道



6.2 I/O设备和设备控制器

□ I/O设备一般由**电子和机械**以下两部分构成：

- 一般I/O设备（物理设备）：机械部分，负责执行I/O操作；
- 设备控制器或适配器(Adapter)：电子部件，负责控制I/O；

□ CPU不直接与I/O通信，是通过**设备控制器**和**通道**。

- **设备控制器**：完成设备与主机间的连接和通讯，在小型和微型机中常采用**印刷电路板**插入计算机中（接口），如显卡、网卡等。
- **通道**：在有的大、中型计算机系统中，还配置了I/O通道或I/O处理机。



6.2.1 I/O设备

1. I/O设备的类型

1) 按使用特性分类:

- 存储设备：外存、辅存
- I/O设备：输入设备、输出设备和交互式设备

2) 按传输速率分类

- 低速设备：每秒几个至数百个字节，如鼠标、键盘
- 中速设备：每秒数千至数十万个字节，如激光打印机
- 高速设备：每秒数十万至千兆字节，如磁盘、磁带、光盘



1. I/O设备的类型（补）

3) 按信息交换方式分类

- 块设备：以数据块为单位存储、传输信息。其基本特征是传输速较高、可寻址，可随机读/写任意一块。如磁盘，常采用DMA方式。
- 字符设备：以字符为单位存储、传输信息。如键盘、打印机。其特点是传输速率低、不可寻址，常采用I/O中断方式。



1. I/O设备的类型（补）

4) 按资源分配角度分类：

- 独占设备：互斥共享，如打印机、磁带。所有字符设备都是独享设备。
- 共享设备：同一段时间内可有多个进程共同使用的设备，如磁盘。
- 虚拟设备：通过SPOOLing技术将独占设备改造为共享设备，讲一个物理设备变为多个逻辑设备。



6.2.1 I/O设备

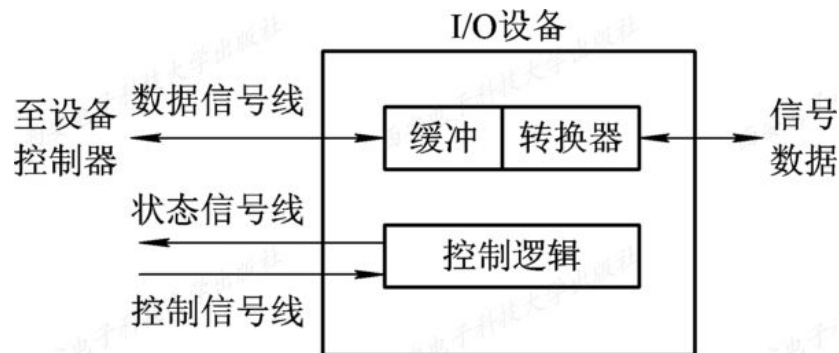
2. 设备控制器与设备的接口

通常，I/O设备通过**设备控制器**与CPU进行通信，因此，在I/O设备中应含有与设备控制器间的接口，在该接口中有三种类型的信号，各对应一条信号线。

(1) 数据信号线：传送数据

(2) 控制信号线：规定了设备要执行的操作，如读、写

(3) 状态信号线：指示设备当前的状态，如正在读（或写）





6.2.2 设备控制器

- ❑ **设备控制器**是CPU和I/O设备之间的**接口**，负责接收CPU发来的命令，并控制I/O设备工作，使处理机能从繁杂的设备控制事务中解脱出来。**可编址**。
- ❑ 设备控制器的主要功能：**控制一个或多个I/O设备，以实现I/O设备和计算机之间的数据交换**。
- ❑ 设备控制器分为**两**类：
 - 用于控制字符设备的控制器；
 - 用于控制块设备的控制器。



6.2.2 设备控制器

1. 设备控制器的基本功能

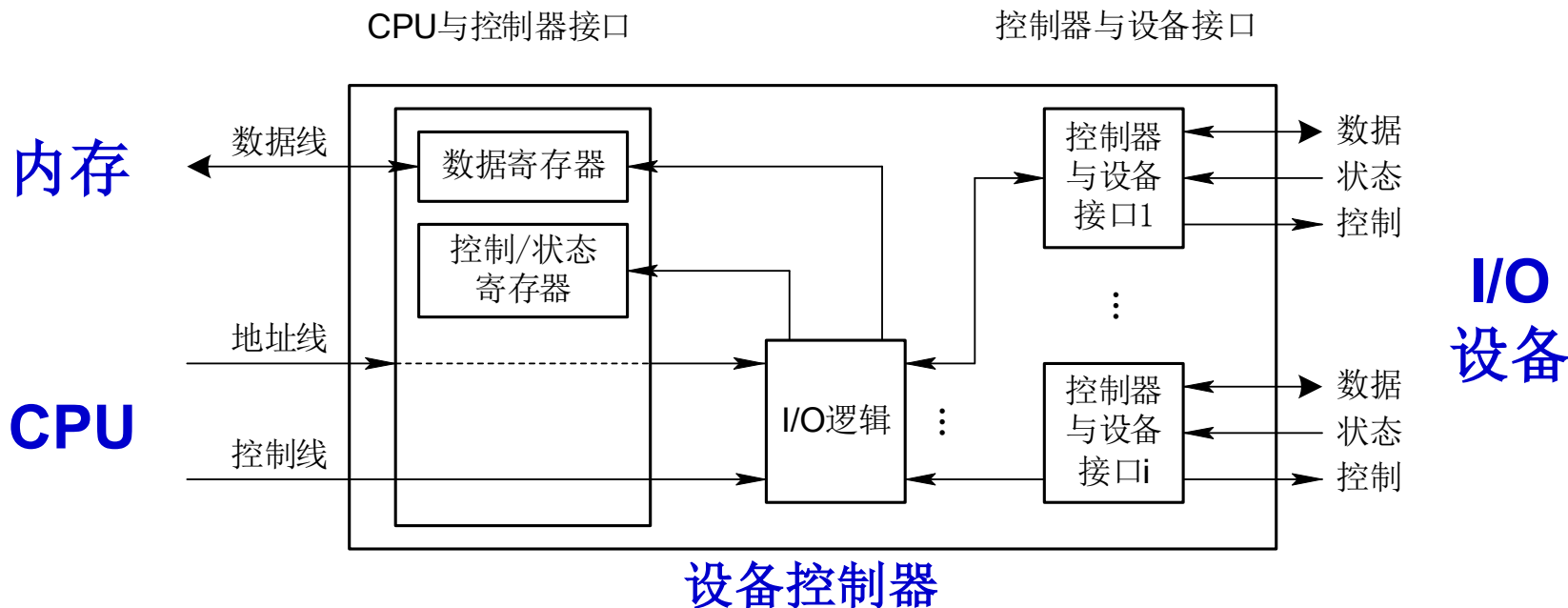
- (1) 接收和识别命令：**控制寄存器**存放CPU发来的命令和参数并译码；
- (2) 数据交换：**CPU**和设备控制器之间、设备控制器和设备之间的数据交换；
- (3) 标识和报告设备的状态：**状态寄存器**存放设备状态信息；
- (4) 地址识别：由I/O逻辑完成；
- (5) 数据缓冲区：暂存数据；
- (6) 差错控制



2. 设备控制器的组成

现有的大多数控制器都是由以下三部分组成：

- (1) 设备控制器与处理机的接口。
- (2) 设备控制器与设备的接口。
- (3) I/O逻辑。

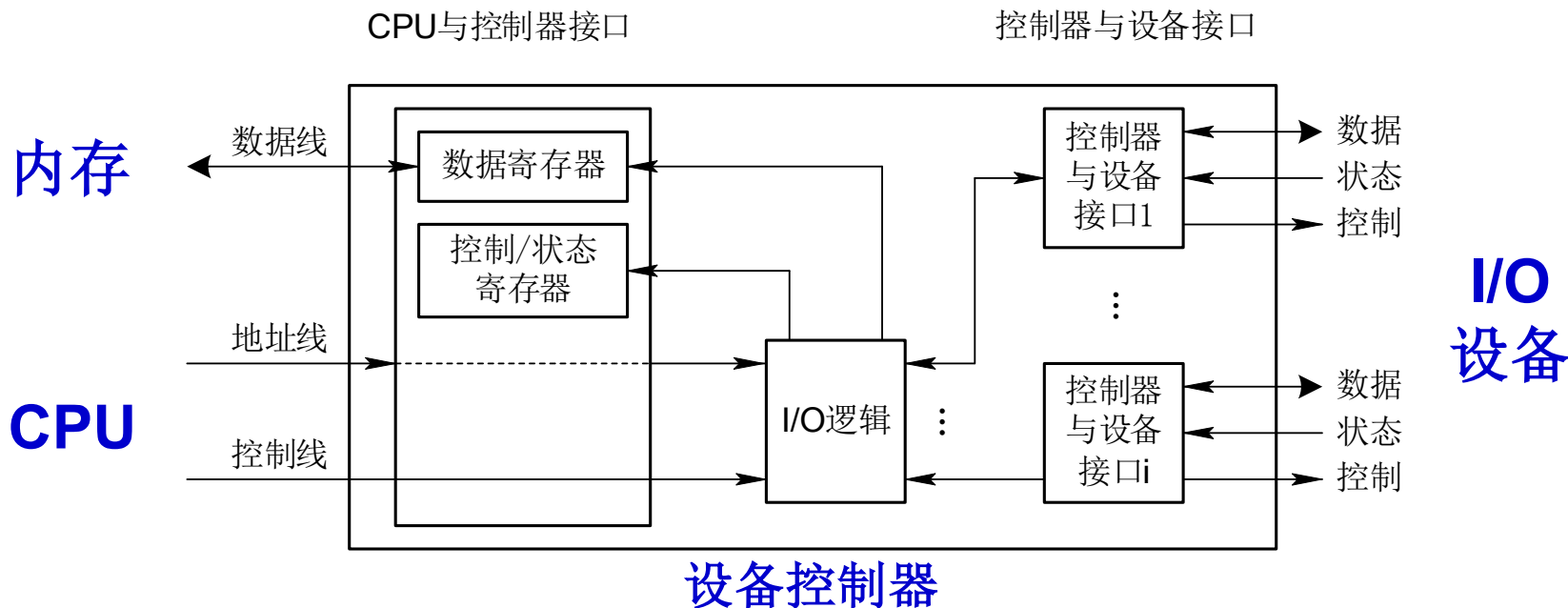




2. 设备控制器的组成

(1) 设备控制器与处理机的接口

- 数据寄存器：暂存来自I/O设备的数据；
- 控制寄存器：存放CPU发来的命令和参数；
- 状态寄存器：存放设备执行结果和状态信息，供CPU读取





2. 设备控制器的组成

(3) I/O逻辑

I/O逻辑用于实现对设备的控制。每当CPU启动设备时，将启动命令发给控制器，同时通过地址线将地址发给控制器，由控制器的I/O逻辑对地址进行译码，并对所选设备进行控制。



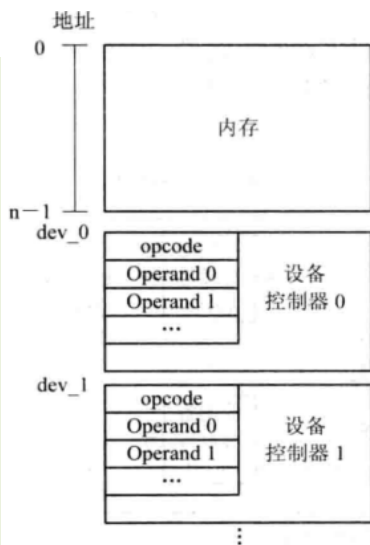
6.2.3 I/O端口编址方式

CPU通过I/O端口地址访问设备，I/O端口有两种编址方案：

1. 利用特定的I/O指令（独立编址）

早期计算机中，为**每个控制寄存器分配一个I/O端口**，**每个端口分配一个I/O端口号**，**独立编址**，同时设置一些**特定指令**。

优点：I/O端口数比主存单元少得多，只需少量地址线，寻址速度快；使用专用I/O指令，程序更清晰。



(a) 采用特定的指令形式

缺点：访问主存和I/O需要两种不同指令，控制复杂，灵活性差。

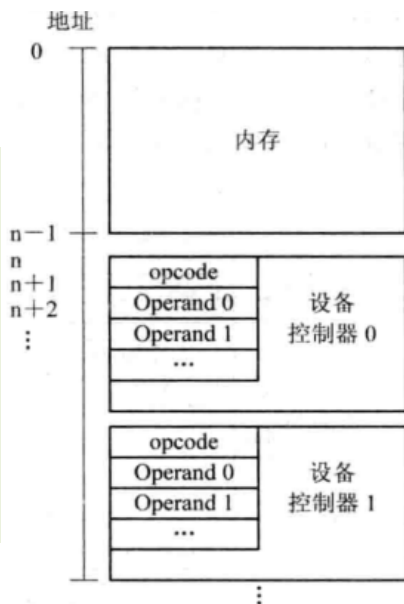


6.2.3 I/O端口编址方式

2. 内存映像I/O（又称统一编址）

将设备控制器与内存统一编址（如：0~n-1被认为是内存地址，大于n为控制器寄存器地址）、统一指令。

优点：不需要专门的I/O指令，使得CPU访问I/O更灵活和方便；I/O端口的编址空间大。



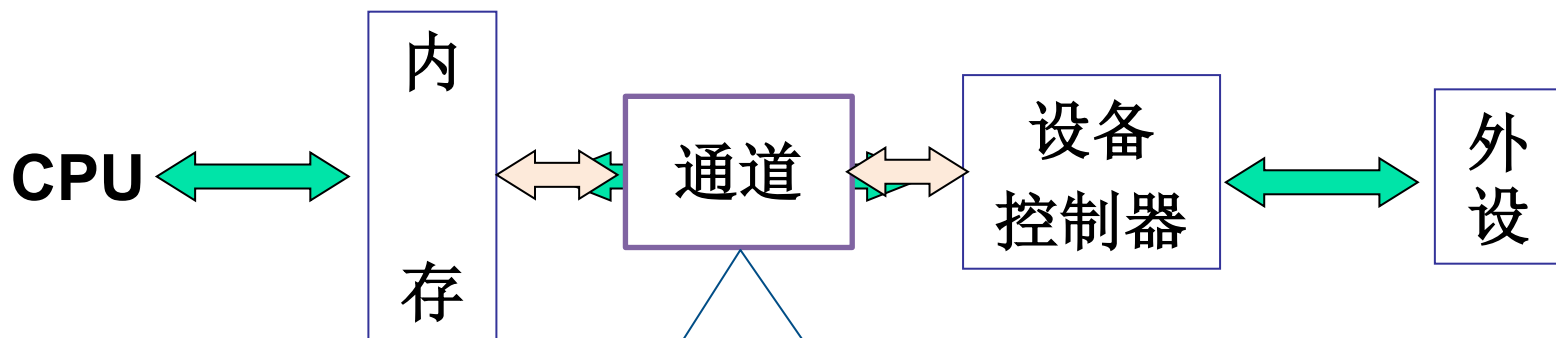
(b) 内存映像I/O形式

缺点：I/O端口地址线条数多，译码电路复杂，寻址速度慢。



6.2.4 I/O通道

1. I/O通道设备的引入



特殊的处理机，能执行I/O指令，并控制I/O操作。



1. I/O通道设备的引入

- **I/O通道**是**独立于CPU**的专门负责数据输入/输出工作的一种特殊的**处理机**，具有执行I/O指令的能力，并通过执行通道程序控制I/O操作。
- 引入通道的目的：
 - (1) 原来由CPU处理的I/O任务转由通道完成，通过执行通道程序来控制I/O操作，使CPU从I/O事务中解脱出来。
 - (2) 提高CPU与设备、设备与设备之间的并行工作能力。
- **引入通道后的I/O操作过程**：CPU向通道发送一条I/O指令，通道收到指令后，从内存中取出本次要执行的通道程序，并执行，完成规定I/O任务后，通道向CPU发中断信号。



1. I/O通道设备的引入

□ 通道与CPU的异同点：

- 有运算和控制逻辑
- 功能较弱、速度较慢
- 有累加器和寄存器
- 有专门的指令系统
- 有向内存直接存取数据的能力
- 指令单一（与I/O操作的有关指令）
- 没有自己的内存，它与CPU共享内存

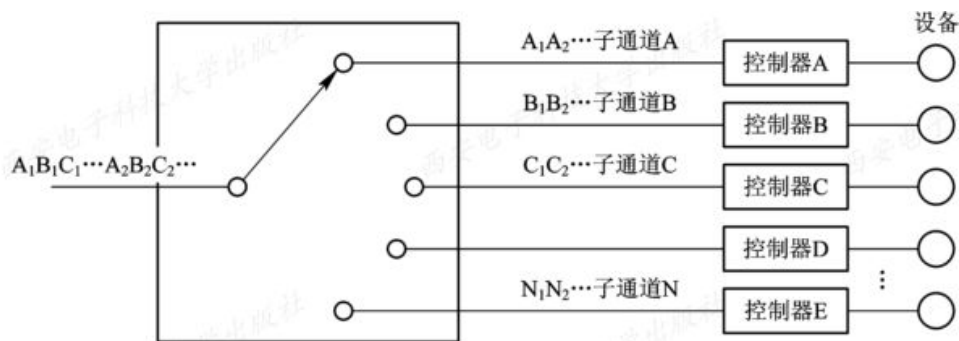
只要字节多路通道扫描每个子通道的速率足够快，而连接到子通道上的设备的速率又不是太高，便不会丢失信息。

2. 通道类型

根据信息交换的方式，通道可以分为以下三类：

□ 字节多路通道(Byte Multiplexor Channel) 不适于高速设备

- 按**字节交叉方式**工作，可以分时地执行多个子通道程序，这些子通道按**时间片轮转方式**共享主通道。
- 以**字节**为单位传输信息，当一台设备传送一个字节后，硬件控制去执行另一个子通道程序，为另一台设备传送字节。
- 主要连接以字节为单位的低速I/O设备。如打印机，终端。





2. 通道类型

□ 数组选择通道(Block Selector Channel) 利用率低

- 以**成组方式**工作，即每次传送**一批数据**，故传送速度很高。
- 一段时间内只能执行一个通道程序，控制一台设备进行数据传送；当某台设备占用该通道后，便一直由它独占，即便它无数据传送，也不允许其他设备使用该通道，直至该设备传送完毕释放该通道。
- 一旦分配，就被独占，利用率较差。
- 主要连接磁盘，磁带等高速I/O设备。



2. 通道类型

□ 数组多路通道(Block Multiplexor Channel)

- 结合了选择通道传送速度快和字节多路通道能进行分时并行操作的优点。
- 先为一台设备执行一条通道指令
- 再自动转接，为另一台设备执行一条通道指令
- 主要连接高速设备



3. 瓶颈问题

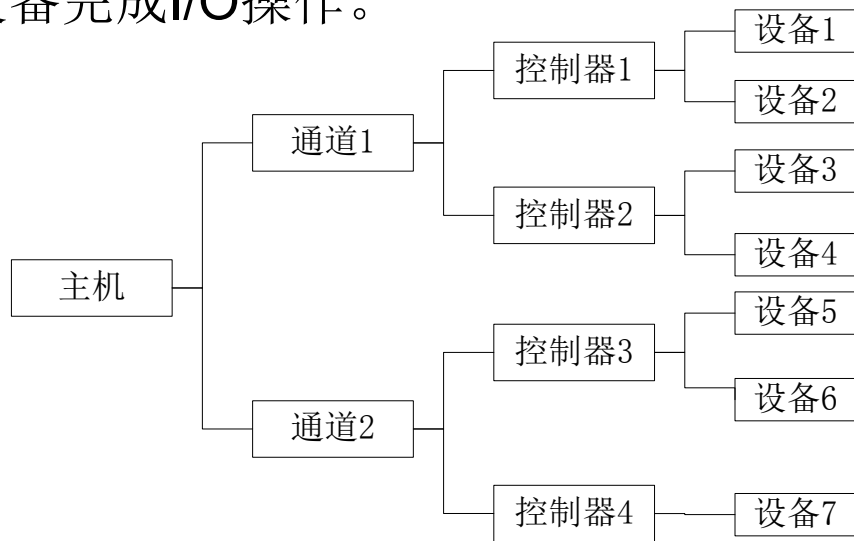
□ 主机系统的三层结构

- 现代计算机I/O系统结构由**通道**、**设备控制器**和**设备**组成。
- I/O操作要经过三级控制：

第一级：CPU发送I/O指令，查询通道状态，启动或停止通道运行；

第二级：通道收到I/O指令后，执行相应通道程序，向设备控制器发命令；

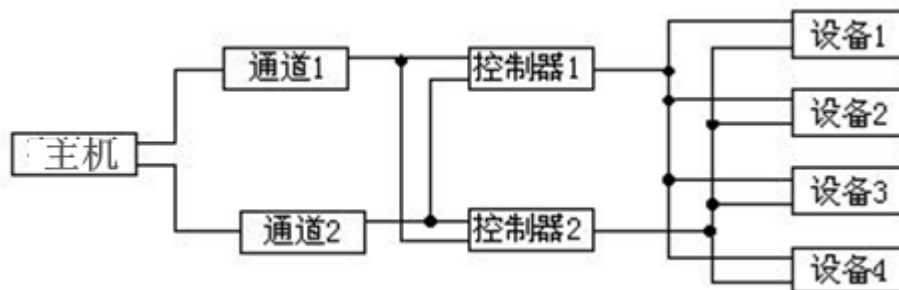
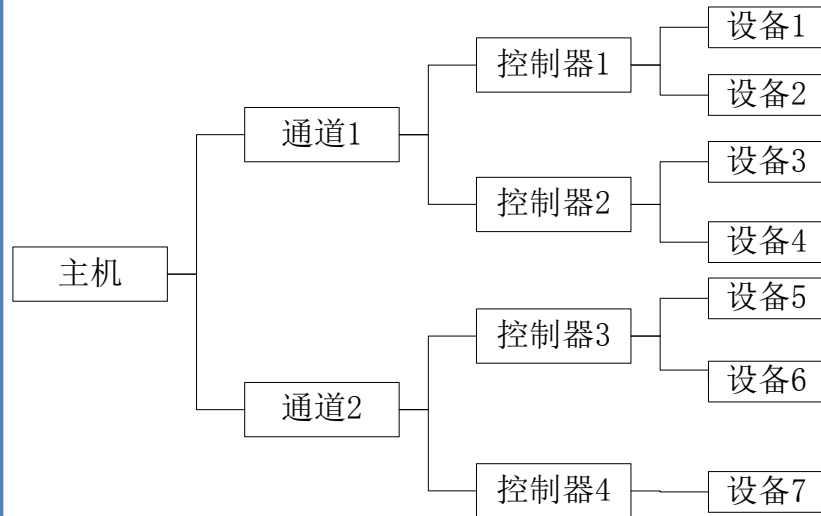
第三级：由设备控制器控制设备完成I/O操作。





3. 瓶颈问题

单通路I/O系统（如图6-7），为了启动设备4，必须用通道1和控制器2，但若这两者已被设备3占用，必然无法启动设备4。这就是由于通道不足而造成的“瓶颈”现象。



解决办法：增加设备到主机间的通路而不增加通道。



6.3 中断机构和中断处理程序

□ 6.3.1 中断简介

□ 6.3.2 中断处理程序



6.3 中断机构和中断处理程序

中断在操作系统中有着特殊重要的地位：

- 中断是**多道程序**得以实现的基础，进程之间的切换是通过中断来完成的。
- 中断也是**设备管理**的基础，为了提高处理机的利用率和实现CPU与I/O设备并行执行，也必需有中断的支持。
- 中断处理程序是I/O系统中最低的一层，它是**整个I/O系统**的基础。



6.3.1 中断简介

1. 中断和陷入

- **中断**：指CPU对I/O设备发来的中断信号的一种响应。由外部设备或人工干预引起的中断称为**外中断**。如时钟中断、I/O中断等。
- **陷入（trap）**：由CPU内部的事件而引发的中断。如非法指令、地址越界、缺页等等，也称为**内中断或异常**。

中断和陷入的主要区别是信号来源是来自CPU外部还是内部。



6.3.1 中断简介

中断号	入口地址
12	8000
14	9001

2. 中断向量表和中断优先级

□ 中断向量表：存放**设备中断处理程序的入口地址**。

- 通常为每种设备配以相应的**中断处理程序**，并把该程序的**入口地址放在中断向量表的一个表项中**；
- 为每个设备的中断请求规定一个**中断号**，它直接对应于中断向量表的一个表项；
- 当I/O设备发来中断请求信号时，由中断控制器确定该请求的中断号，根据设备中断号查找中断向量表，中断处理程序的入口地址，然后执行中断处理程序。

□ 中断优先级：系统需要为设备规定不同的优先级。



6.3.1 中断简介

3. 对多中断源的处理方式

处理机正在处理一个中断时，对新中断请求有两种处理方式：

- ❑ **屏蔽（禁止）中断**：处理机处理一个中断时，完成前屏蔽所有中断。优点是简单，但不能用于实时性要求较高的中断请求。
- ❑ **嵌套中断**：常用于在设置中断优先级的系统中：
 - 同时有多个不同优先级的中断请求时，CPU优先响应**最高**优先级的中断请求；
 - **高**优先级的中断请求**可以抢占**正在运行的**低**优先级中断的处理机。



6.3.2 中断处理程序

中断处理程序的处理过程：

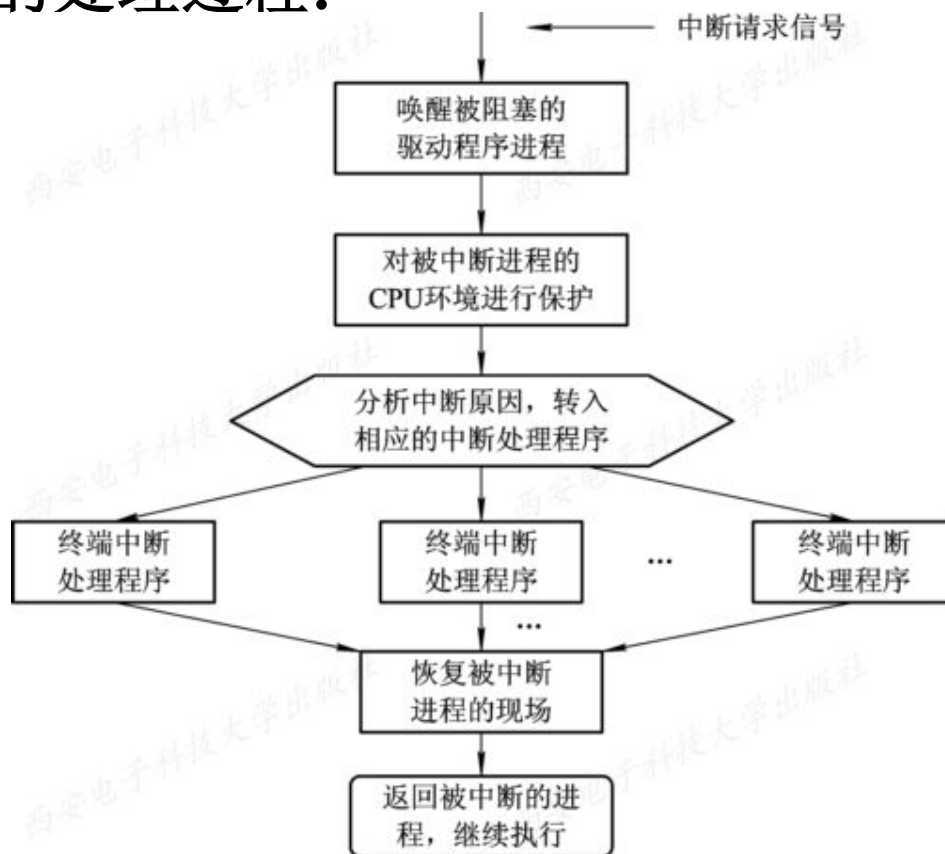


图6-11 中断处理流程



6.4 设备驱动程序

- 6.4.1 设备驱动程序概述

- 6.4.2 设备驱动程序的处理过程

- 6.4.3 对I/O设备的控制方式



6.4 设备驱动程序

- ❑ 设备驱动程序：通常又称为设备处理程序，是I/O系统的高层与设备控制器之间的通信程序。
- ❑ 主要任务：接收上层软件（设备独立性软件）发来的抽象I/O要求，如read或write命令，转换为具体要求后，发送给设备控制器，启动设备去执行；反之，将由设备控制器发来的信号传送给上层软件。
- ❑ 设备驱动程序与硬件密切相关，通常为每一类设备配置一种驱动程序。例如，打印机和显示器需要不同的驱动程序。



6.4.1 设备驱动程序概述

1. 设备驱动程序的功能

(1) 接收上层软件（设备独立性软件）发来的命令和参数，并将抽象要求转换为与设备相关的低层操作序列。

(2) 检查用户I/O请求的合法性，了解I/O设备的工作状态，传递与I/O设备操作有关的参数，设置设备的工作方式。

(3) 发出I/O命令，如果设备空闲，便立即启动I/O设备，完成指定的I/O操作；如果设备忙碌，则将请求者的请求块挂在设备队列上等待。

(4) 及时响应由设备控制器发来的中断请求，并根据其中断类型，调用相应的中断处理程序进行处理。



6.4.1 设备驱动程序概述

2. 设备驱动程序的特点

设备驱动程序属于低级的系统例程，与一般应用程序及系统程序有下述明显差异：

(1) 驱动程序是实现**设备独立性软件和设备控制器之间通信和转换**的程序，具体说，它将抽象的I/O请求转换成具体的I/O操作后传送给控制器。又把控制器中所记录的设备状态和I/O操作完成情况，及时地反映给请求I/O的进程。

(2) 驱动程序与设备控制器以及I/O设备的硬件特性紧密相关，对于**不同类型的设备，应配置不同的驱动程序**。但可以为**相同的多个终端设置一个终端驱动程序**。



6.4.1 设备驱动程序概述

2. 设备驱动程序的特点

(3) 驱动程序与I/O设备所采用的I/O控制方式紧密相关，常用的I/O控制方式是中断驱动和DMA方式。

(4) 由于驱动程序与硬件紧密相关，因而其中的一部分必须用汇编语言书写。目前有很多驱动程序的基本部分已经固化在ROM中。

(5) 驱动程序应允许可重入。一个正在运行的驱动程序常会在一次调用完成前被再次调用。



6.4.1 设备驱动程序概述

3. 设备处理方式

(1) 为**每一类**设备设置**一个进程**，专门用于执行这类设备的I/O操作。这种方式比较适合于**较大**的系统。

(2) 在**整个系统**中设置**一个I/O进程**，专门用于执行系统中所有各类设备的I/O操作。也可以设置一个输入进程和一个输出进程，分别处理系统中的输入或输出操作。

(3) 不设置专门的设备处理进程，而**只为各类设备设置相应的设备驱动程序**，供用户或系统进程调用。这种方式目前用得较多。



6.4.2 设备驱动程序的处理过程

设备驱动程序的主要任务是**启动指定设备，完成上层指定的I/O工作**。但在启动之前，应先完成必要的准备工作，如检测设备状态是否为“忙”等。在完成所有的准备工作后，才向设备控制器发送一条启动命令。设备驱动程序的处理过程如下：

- (1) 将抽象要求转换为具体要求。
- (2) 对服务请求进行校验。
- (3) 检查设备的**状态**。（状态寄存器中的格式）
- (4) 传送必要的参数。
- (5) 启动I/O设备。



6.4.3 对I/O设备的控制方式

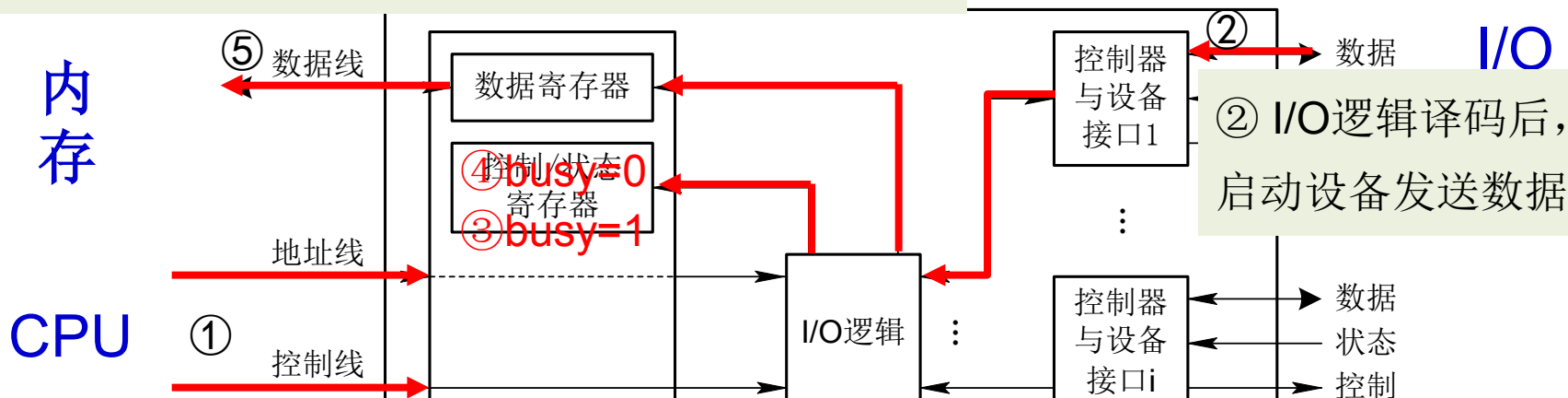
- **I/O控制**是指控制**I/O设备和主机**（CPU和内存）之间的数据传输。
- **I/O控制方式发展宗旨**：尽量**减少CPU对I/O控制的干预**，把CPU从繁杂的I/O控制事务中解脱出来，以便更多地去完成数据处理任务。
- **I/O控制方式**：
 1. 使用轮询的可编程I/O方式
 2. 使用中断的可编程I/O方式
 3. 直接存储器访问控制(**DMA**)
 4. I/O通道控制方式



1. 使用轮询的可编程I/O方式 改进：引入中断技术

- ❑ 最早期的一种I/O控制方式，又称**程序直接控制**或**忙-等待方式**。
- ❑ 工作原理：CPU向设备控制器发出一条指令后，启动I/O设备，并不断**循环检测设备状态**，直到确定数据（字或字节）已在设备控制器的数据寄存器中。以CPU读I/O操作为例：

⑤ CPU取出数据寄存器DR中的数据，送入内存



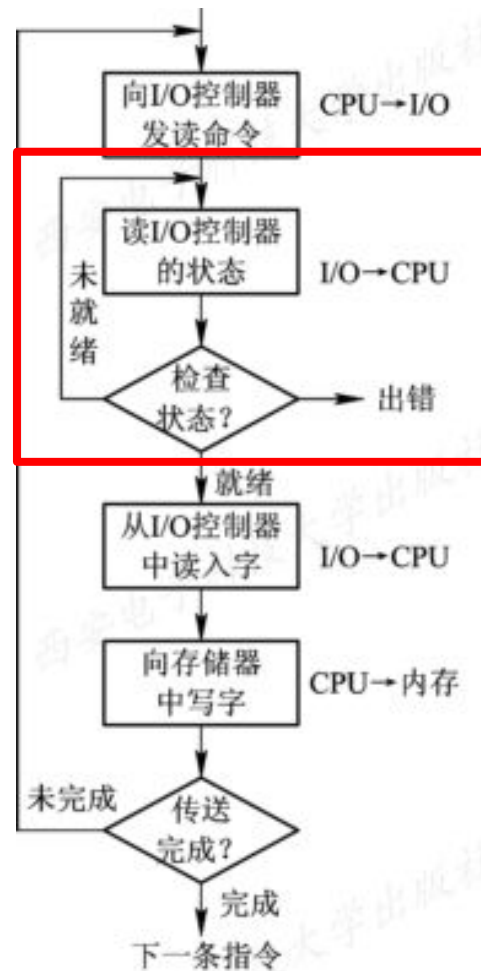
① CPU向控制器发出读I/O指令，同时通过地址线向控制器发地址

③ 同时置状态寄存器的busy=1 ④ CPU一直在检测busy，直至其值为0。



1. 使用轮询的可编程I/O方式

- 优点：易实现
- 缺点：但CPU绝大部分时间用于循环测试I/O设备状态中，处于“忙等”状态；CPU和I/O只能串行工作，CPU利用率低。



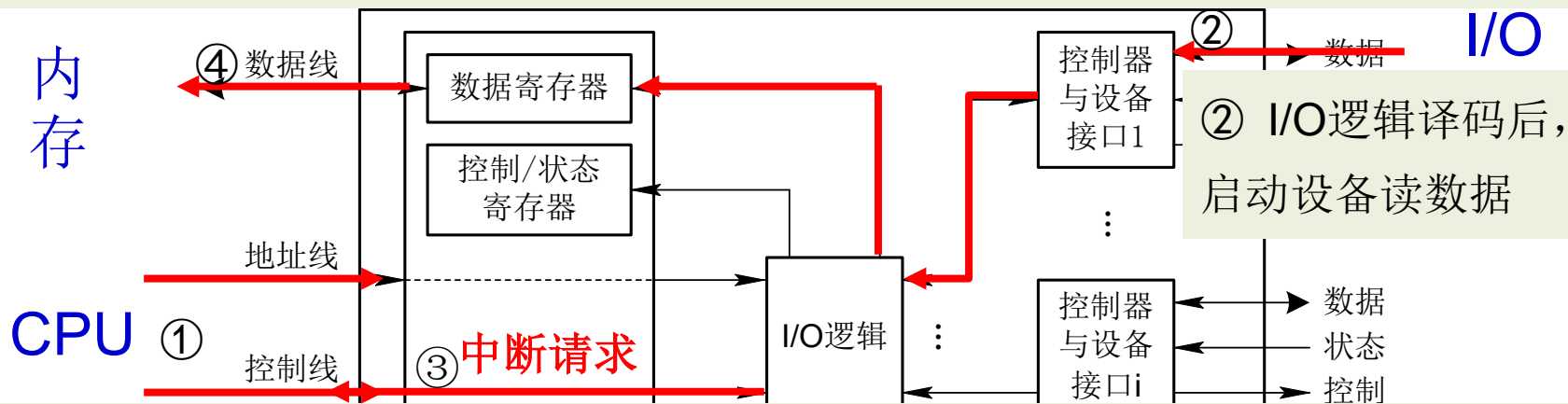
(a) 程序I/O方式



2. 使用中断的可编程I/O方式 以字或字节为单位

□ 工作原理：CPU发送I/O命令后，阻塞当前进程，继续执行其他进程。当数据进入数据寄存器后，设备控制器发送中断信号告知CPU。以CPU读I/O操作为例：

④ CPU收到中断信号后，暂停正在执行的进程，响应中断，唤醒阻塞进程，取出数据寄存器中的数据，送入内存

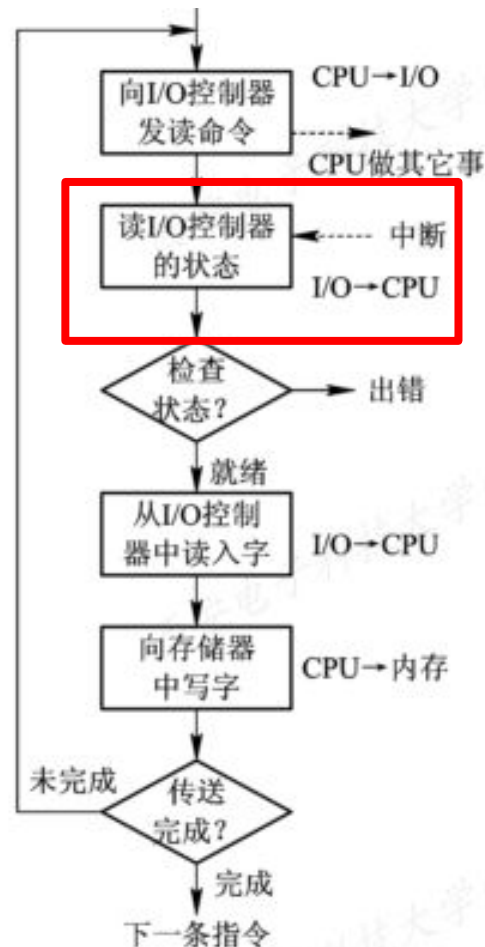


① CPU向控制器发出读I/O指令，同时通过地址线向控制器发地址。阻塞当前进程，继续执行其他进程。 ③ 数据进入数据寄存器后，向CPU发送一个中断。



2. 使用中断的可编程I/O方式

- CPU会在每个指令周期的末尾检查是否有中断；
- 中断处理过程需要保存、恢复进程的运行环境，这个过程需要一定的时间开销。可见，如果中断发生的频率太高，会降低系统性能。

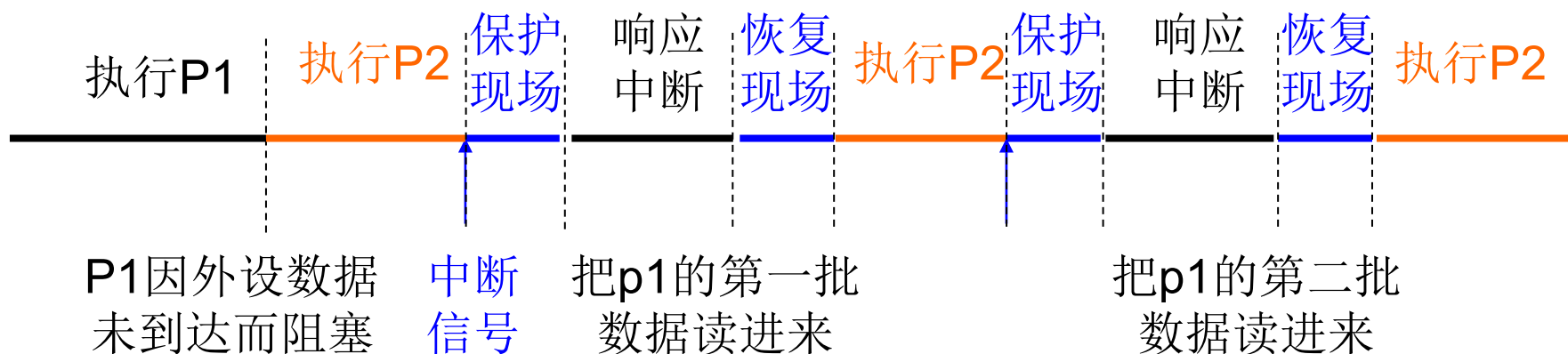


(b) 中断驱动I/O方式



2. 使用中断的可编程I/O方式 以字或字节为单位

- 优点：实现CPU与I/O设备并行工作。仅当I/O准备好数据时，才需CPU花费极短的时间进行中断处理。提高了整个系统的资源利用率及吞吐量。

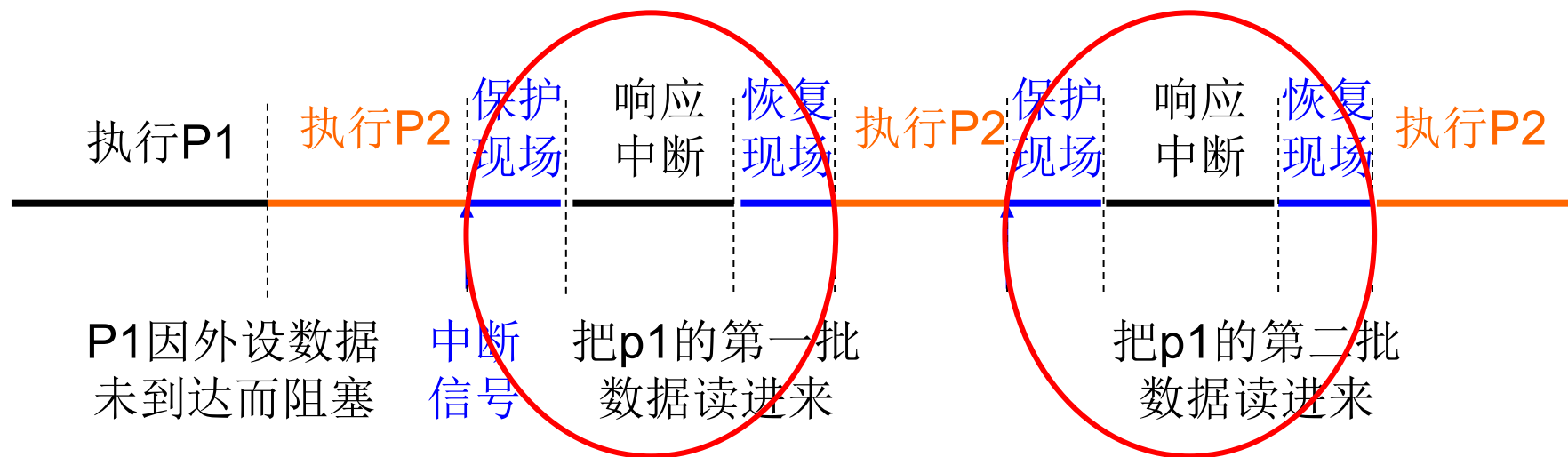


- 缺点：若传输大批数据，每传送几个字节（即数据寄存器每满一次），就请求一次中断，CPU需花费大量开销处理频繁的I/O中断。



3. 直接存储器访问控制(DMA)

1) 直接存储器访问 (DMA) 方式的引入



I/O的数据送入内存

解放CPU，交给专业人士解决

DMA控制器

在I/O和内存之间开辟直接数据交换通路



3. 直接存储器访问控制(DMA)

1) 直接存储器访问 (DMA) 方式的引入

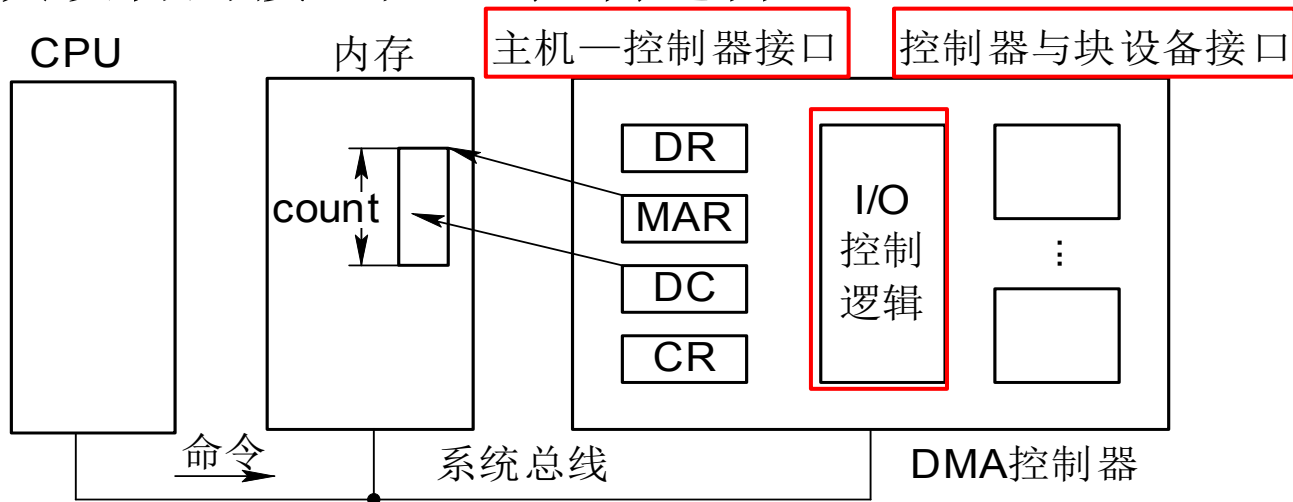
- 数据传输单位是**数据块**，数据由**I/O设备直接送入内存**，或者相反。DMA方式主要用于块设备的I/O控制。
- 仅在传送一个或多个数据块的开始（CPU发命令，启动DMA）和结束（处理DMA发来的中断请求）时，才需要CPU干预；
- **DMA控制器**用于控制主存和I/O模块之间的数据交换。



3. 直接存储器访问控制(DMA)

2) DMA控制器的组成

DMA控制器由三部分组成：**主机与DMA控制器接口**；DMA控制器与块设备的接口；I/O控制逻辑。



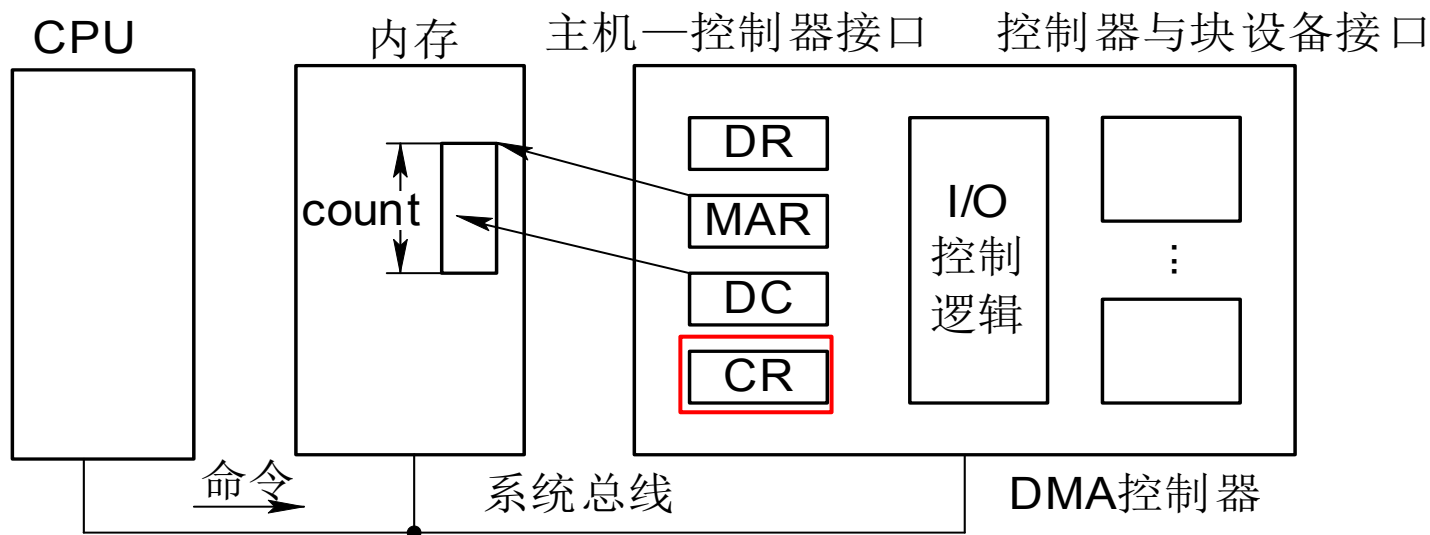
为实现主机和控制器之间直接交换数据块，DMA控制器中设置四类寄存器：CR、MAR、DR、DC。



3. 直接存储器访问控制(DMA)

2) DMA控制器的组成

- 命令/状态寄存器CR: 接收从CPU发来的I/O命令，或有关控制信息，或设备的状态。

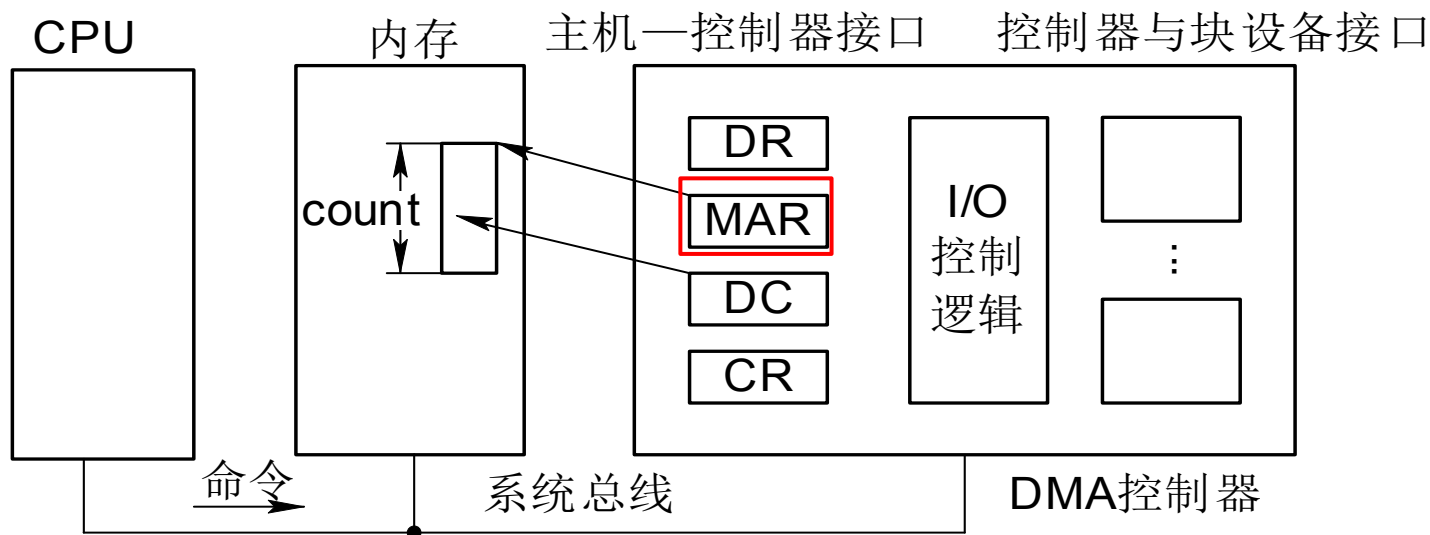




3. 直接存储器访问控制(DMA)

2) DMA控制器的组成

- 内存地址寄存器MAR: 输入时，存放把数据要传送到的内存起始地址，输出时，存放数据在内存中的源地址。

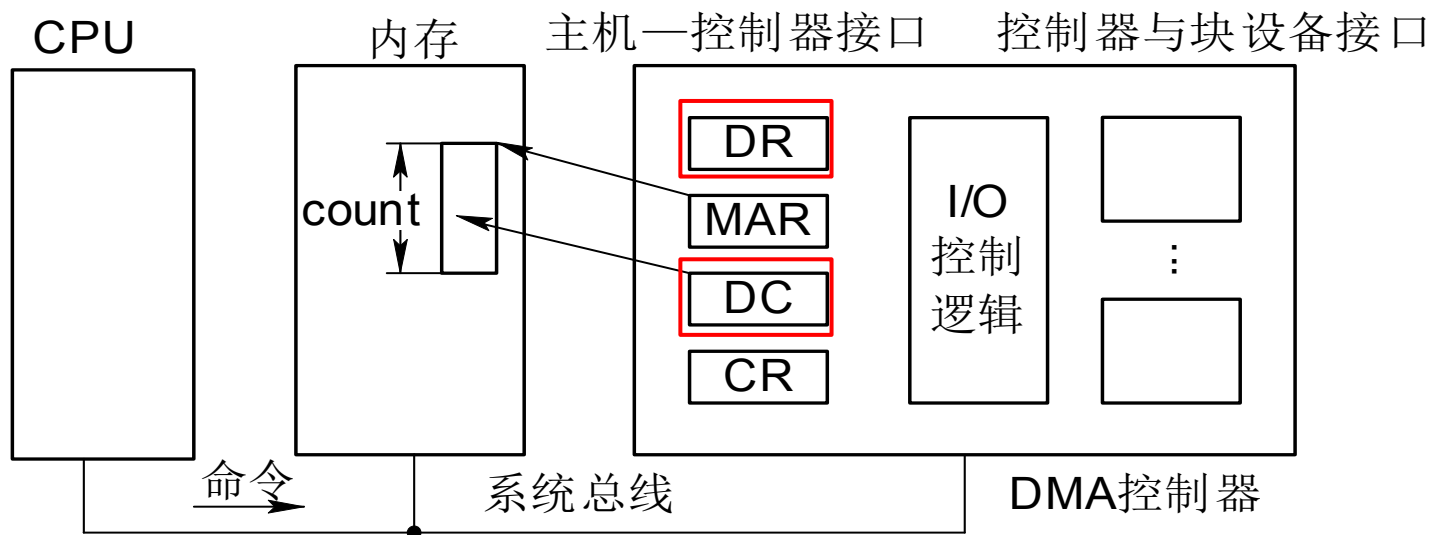




3. 直接存储器访问控制(DMA)

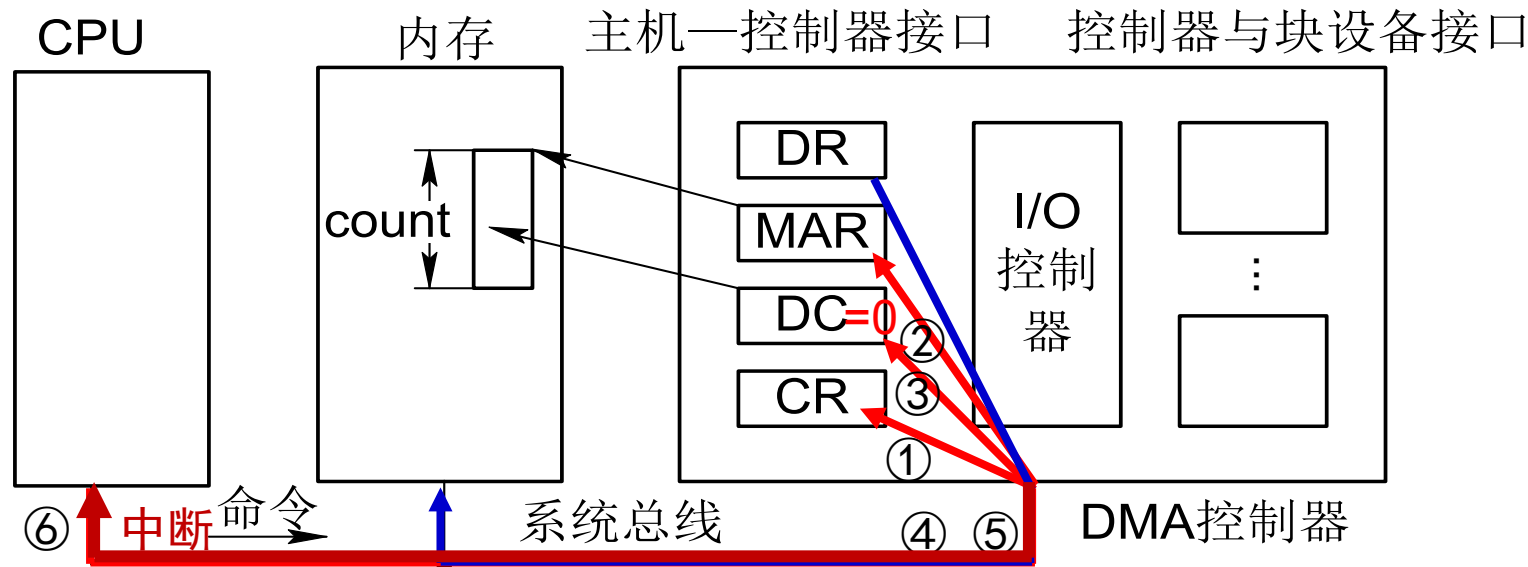
2) DMA控制器的组成

- ❑ 数据寄存器DR：暂存数据。
- ❑ 数据计数器DC：存放本次CPU要读或写的字（节）数。





3) DMA工作过程 以CPU读磁盘数据为例



① CPU向磁盘发送读命令，并将命令送入命令寄存器CR

② CPU将数据要放入的内存起始地址送入内存地址寄存器MAR

③ CPU将要传送的字节数count送入数据计数寄存器DC

④ 启动DMA控制器进行数据传送，阻塞当前进程，CPU可处理其他进程

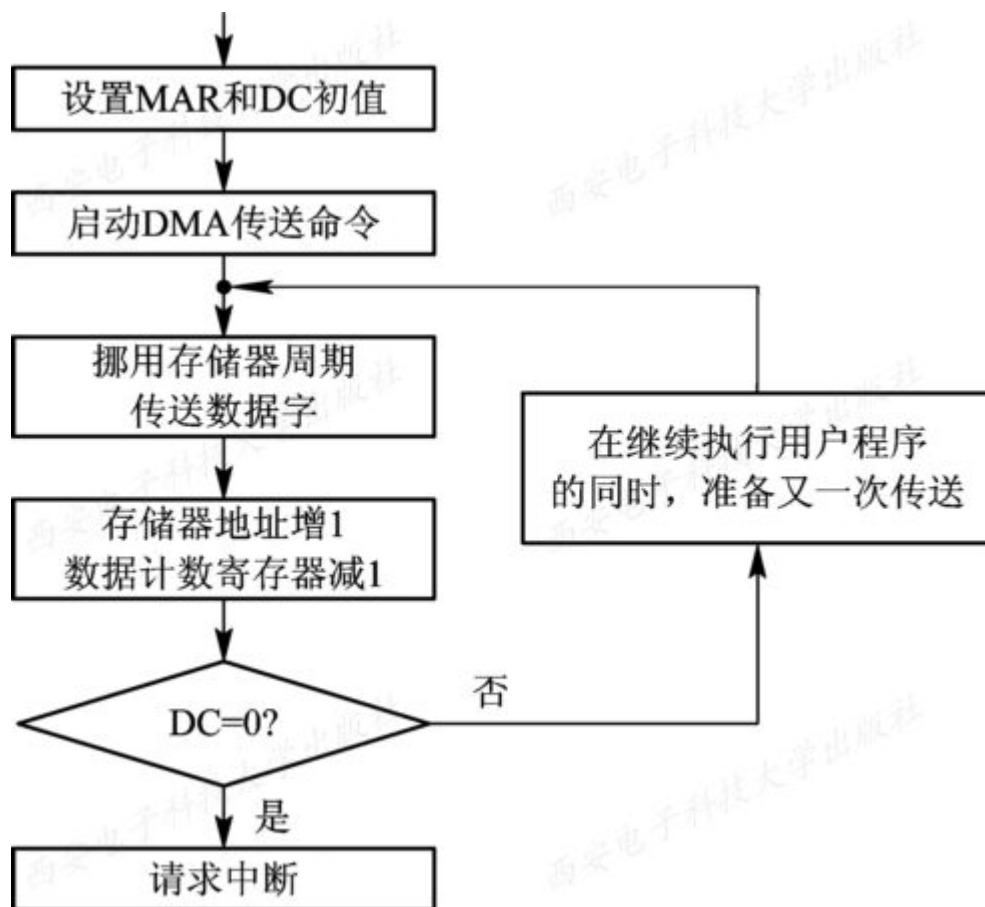
⑤ 在数据块传输过程中，DMA控制器不断把数据通过数据寄存器送到内存

⑥ 当数据块全部传完（DC=0），DMA控制器将向CPU发送一个I/O中断。



3. 直接存储器访问控制(DMA)

3) DMA工作过程





3. 直接存储器访问控制(DMA)

■ 优点:

- DMA控制方式，仅在传送一个数据块的开始和结束时，需要CPU干预。期间数据传送在DMA控制下直接完成。
- 传送一个数据块，仅需一次中断（结束时）即可。

■ 缺点:

- DMA控制器没有自己的总线，所谓的直接完成只是其挪用CPU的工作周期完成。
- 传送多个数据块时，仍需要提出多次中断。



4. I/O通道控制方式

1) I/O通道控制方式的引入

- **I/O通道方式**在DMA基础上，进一步减少CPU的干预，由以数据块为单位的干预减少到以**一组数据块**为单位的干预。
- 当CPU要完成一组相关的读(或写)操作及有关控制时，只需**向I/O通道发送一条I/O指令**，以给出其所要执行的通道程序的首址和要访问的I/O设备，通道接到该指令后，通过执行通道程序便可完成CPU指定的I/O任务。
- 实现了CPU、通道和I/O设备三者的并行操作。

通道：专门负责输入输出操作的处理器。除了具有DMA数据块传输功能，还有**指令系统**，可以实施复杂的输入输出控制。



4. I/O通道控制方式

2) 通道程序

□ 通道程序：由一系列**通道指令**（或称为通道命令）构成。通道指令中都包含下列信息：

- **操作码**：规定指令所执行的操作，如读、写、控制等操作。
- **内存地址**：标明字符送入内存（读操作）和从内存取出（写操作）时的内存首址。
- **计数**：表示本条指令所要读(或写)数据的字节数。
- **通道程序结束位P**：表示通道程序是否结束。**P=1**表示本条指令是通道程序的最后一条指令。
- **记录结束标志R**：**R=0**表示本通道指令与下一条指令所处理的数据是同属于一个记录；**R=1**表示这是处理某记录的最后一条指令。



4. I/O通道控制方式

例：一个由六条通道指令所构成的简单的通道程序。该程序的功能是将内存中不同地址的数据写成多个记录。其中，前三条指令是分别将813~892单元中的80个字符和1034~1173单元中的140个字符及5830~5889单元中的60个字符写成一个记录；第4条指令是单独写一个具有300个字符的记录；第5、6条指令共写含500个字符的记录。

操 作	P	R	计 数	内 存 地 址
WRITE	0	0	80	813
WRITE	0	0	140	1034
WRITE	0	1	60	5830
WRITE	0	1	300	2000
WRITE	0	0	250	1650
WRITE	1	1	250	2720



6.5 与设备无关的I/O软件（设备独立性软件）

- 6.5.1 与设备无关软件的基本概念
- 6.5.2 与设备无关的软件
- 6.5.3 设备分配
- 6.5.4 逻辑设备名到物理设备名映射的实现



6.5.1 与设备无关软件的基本概念

□ 设备独立性（设备无关性）基本含义：应用程序中所用的设备，不局限于使用某个具体的物理设备。

1. 以物理设备名使用设备

使用设备的物理名称申请设备不灵活。

2. 引入了逻辑设备名

使用逻辑设备名，分配灵活；易于实现I/O重定向。

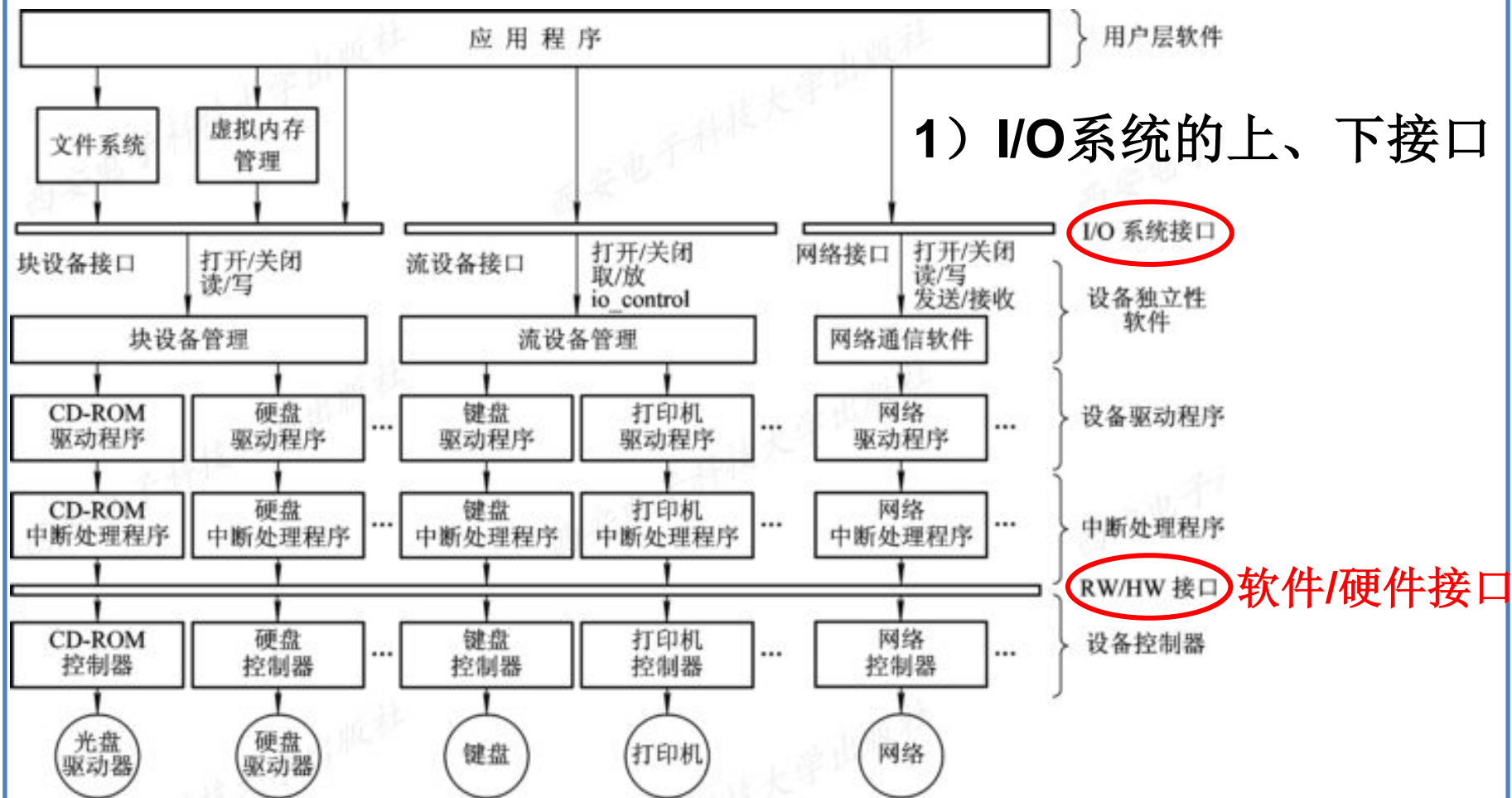
3. 逻辑设备名到物理设备名的转换

在应用程序中，用逻辑设备名来请求使用某类设备；而系统实际执行时，还必须使用物理设备名。因此，需有逻辑设备名→物理设备名的转换功能。



6.1.2 I/O系统的层次结构和模型

2. I/O系统中各种模块之间的层次视图





6.5.2 与设备无关的软件

为了实现设备独立性，必须再在驱动程序之上设置一层软件，称为**与设备无关的软件**，是I/O系统的最高层软件。与设备无关的软件的主要功能包括：

1. 设备驱动程序的统一接口
2. 缓冲管理
3. 差错控制
4. 对独立设备的分配与回收
5. 独立于设备的逻辑数据块

设备驱动程序的统一接口
缓冲
错误报告
分配与释放专用设备
提供与设备无关的块大小



6.5.3 设备分配

2. 设备分配时应考虑的因素

(1) 设备的固有属性

- 独占设备：分给进程后，独占，直至完成或释放该设备；
- 共享设备：可同时分配给多个进程，需合理调度；
- 虚拟设备：可当共享设备对待

(2) 设备分配算法

- 先来先服务：根据进程对设备请求先后，排成一个设备请求队列，先分配给队首；
- 优先级高者优先：根据优先级生成设备请求队列。



6.5.3 设备分配

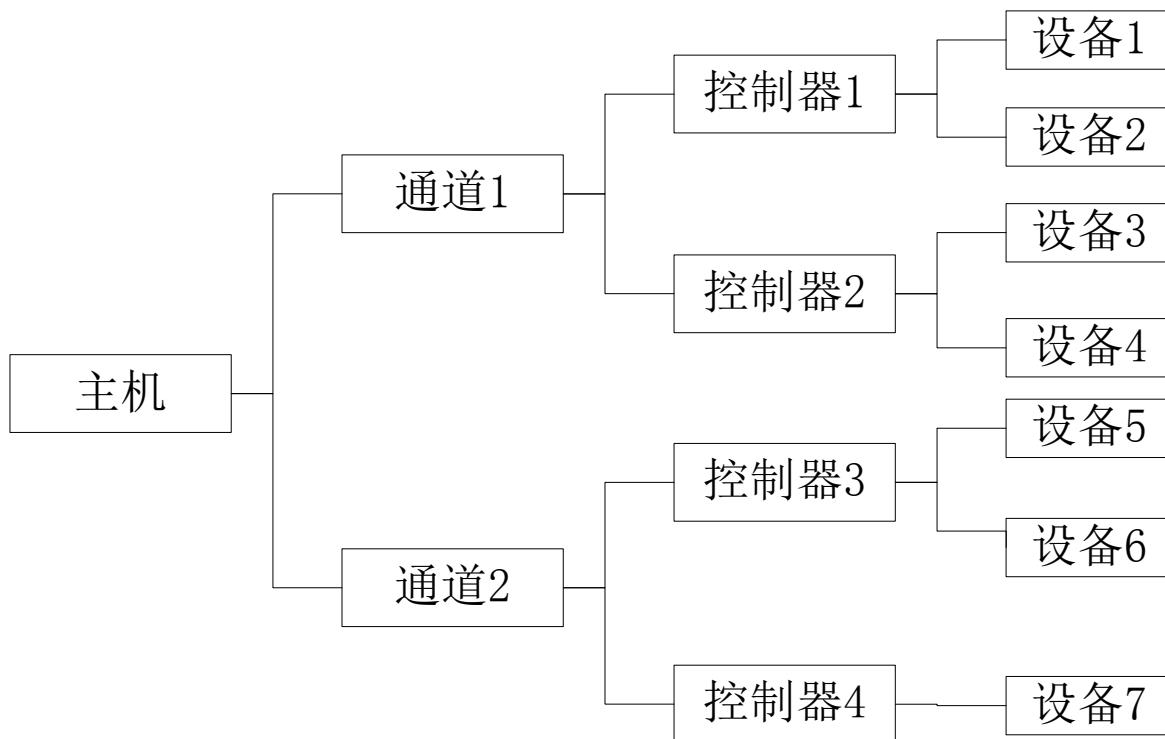
2. 设备分配时应考虑的因素

(3) 设备分配中的安全性

- **安全分配方式：**进程提出I/O请求后即阻塞，直到被唤醒。阻塞时，不请求也不保持资源。优点：设备分配是安全的；缺点是CPU和I/O串行工作，进程进展缓慢。
- **不安全分配方式：**进程发出I/O请求后仍继续运行，需要时又发出新请求。仅当进程所请求的设备已被另一进程占用时，才进入阻塞。优点：一个进程可操作多个设备，进程推进迅速；缺点：可能造成死锁。



6.5.3 设备分配





6.5.3 设备分配

□ 独占设备必须由系统统一分配，系统应配置相应的数据结构。

1. 设备分配中的数据结构

1) 设备控制表DCT：系统为每个设备配置一张DCT，用于记录设备情况。

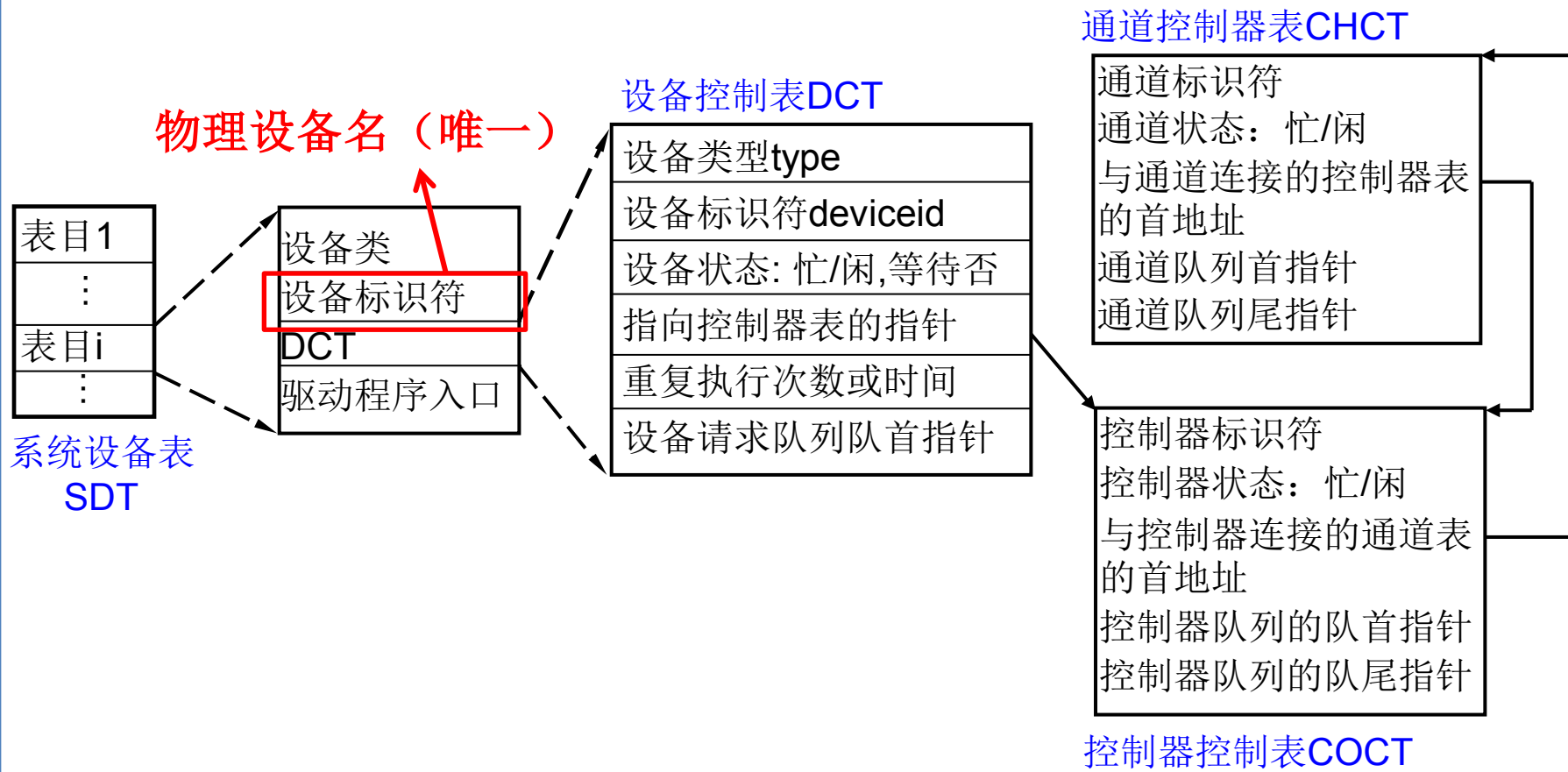
2) 设备控制器控制表COCT：每个设备控制器对应一张COCT，用于记录设备控制器情况。

3) 通道控制表CHCT：每个通道对应一张CHCT，用于记录通道情况。

4) 系统设备表SDT：记录系统中全部设备的情况，每个设备对应一个表项。



1. 设备分配中的数据结构





6.5.3 设备分配

3. 设备分配过程

(1) 基本的设备分配程序

以独占设备为例，分析设备的分配过程。利用SDT、DCT、COCT、CHCT，按下述步骤进行设备分配：

1) 分配设备

根据I/O请求中的**物理设备名**查系统设备表SDT，找出设备DCT。

2) 分配控制器

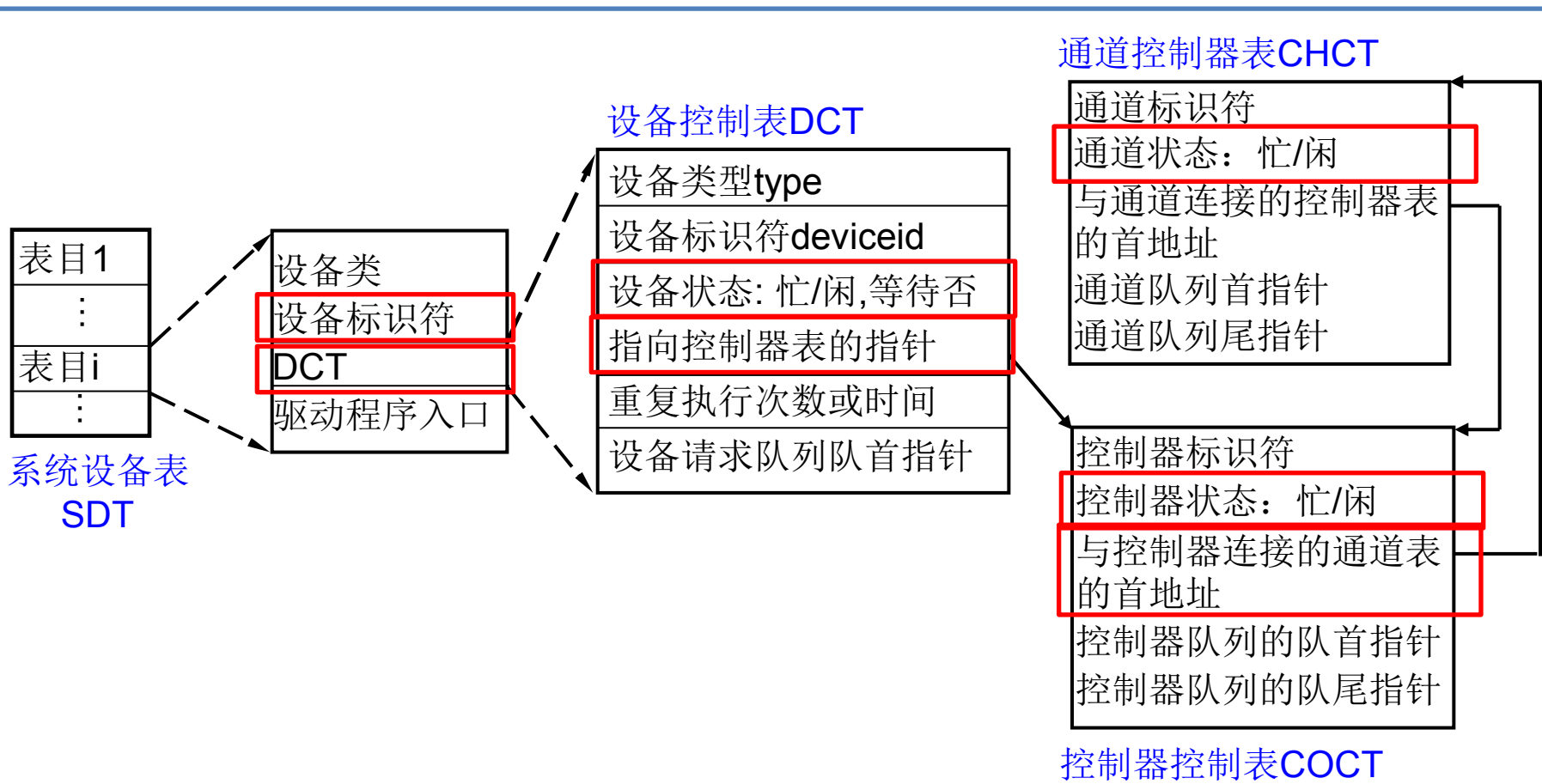
设备分给请求进程后，查DCT找到对应设备控制器的COCT。

3) 分配通道（假设采用的是单通路I/O系统结构）

设备控制器分给请求进程后，查COCT找到对应通道的CHCT。**在设备、设备控制器、通道都分配成功后**，启动I/O进行数据传送。



设备分配





6.5.3 设备分配

3. 以独占设备为例，分析设备的分配过程

(2) 设备分配程序的改进

- 增加设备独立性：**进程应使用逻辑设备名请求I/O；**
- 考虑多通路情况

系统只识别物理设备名（系统设备表DCT中的
设备标识符是物理设备名）



6.5.4 逻辑设备名到物理设备名映射的实现

1. 逻辑设备表LUT

进程用

系统识别

逻辑设备表LUT用于将逻辑设备名映射为物理设备名。每个表目包括：逻辑设备名、物理设备名、设备驱动程序入口地址。

逻辑设备名	物理设备名	驱动程序入口地址
/dev/tty	3	1024
/dev/printer	5	2046
⋮	⋮	⋮

6-19(a)



6.5.4 逻辑设备名到物理设备名映射的实现

2. 逻辑设备表的设置问题

在系统中可采取两种方式设置逻辑设备表：

- 整个系统中只设置一张**LUT**：主要用于单用户系统；
- 每个用户设置一张**LUT**：主要用于多用户系统。



6.6 用户层的I/O软件

- 6.6.1 系统调用与库函数

- 6.6.2 假脱机（SPOOLing）技术



6.6 用户层的I/O软件

一部分I/O软件放在了内核之外的用户层，但仍属于I/O系统的一部分，如库函数、假脱机系统等。

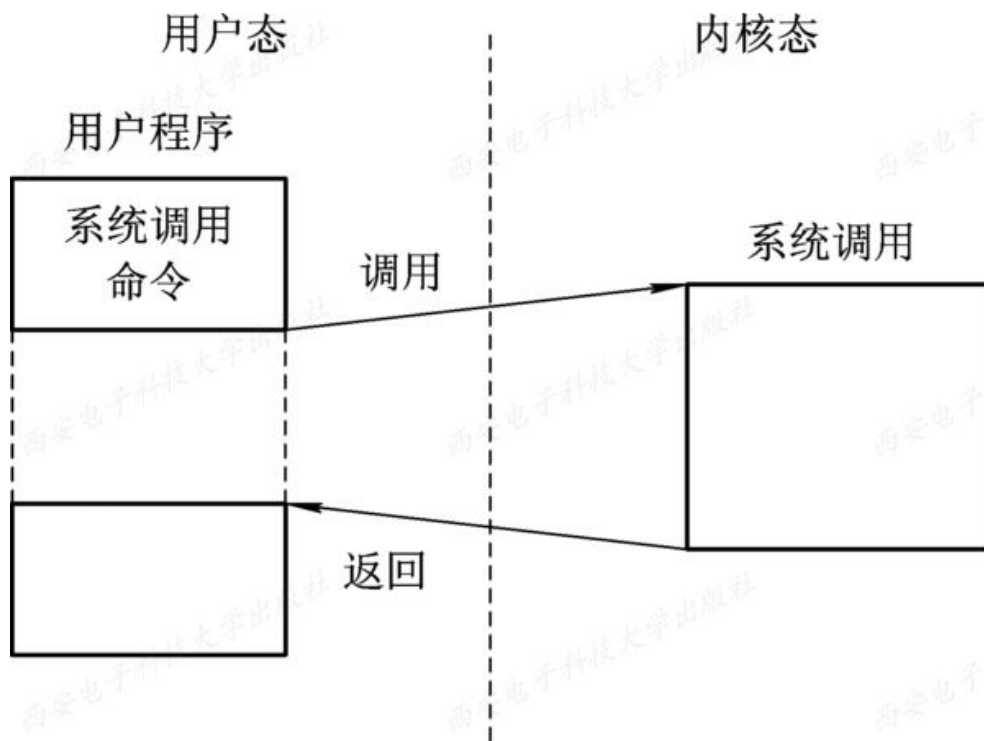
用户层必须通过一组**系统调用**来取得操作系统的服务，在现代的高级语言中，通常提供了与各系统调用一一对应的库函数，用户程序通过调用对应的库函数使用系统调用。



6.6.1 系统调用与库函数

1. 系统调用

应用程序可以通过系统调用间接调用OS中的I/O过程，对I/O设备进行操作。





6.6.1 系统调用与库函数

2. 库函数

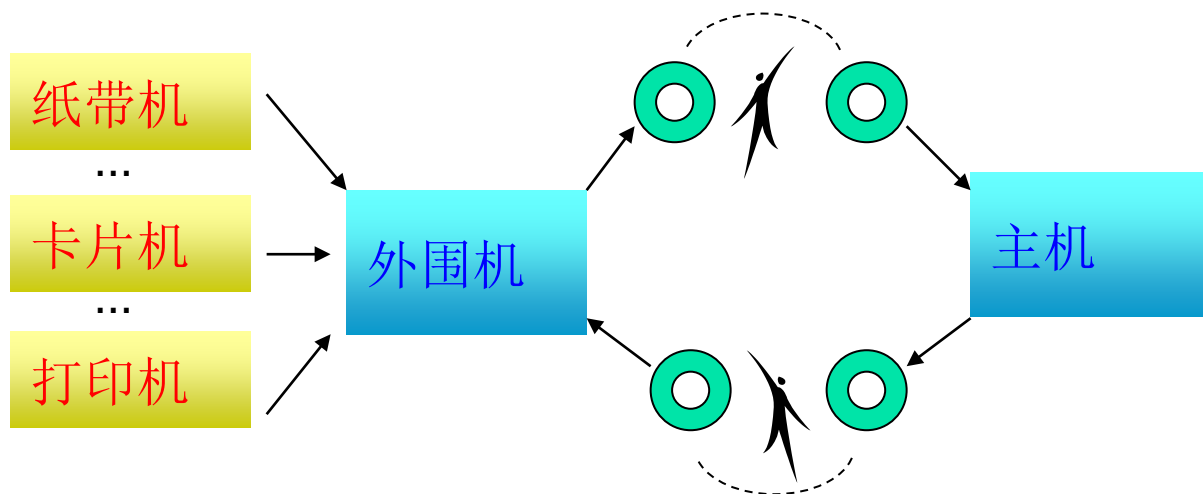
在C语言以及UNIX系统中，系统调用(如read)与各系统调用所使用的库函数(如read)之间几乎是一一对应的。而微软定义了一套过程，称为Win32 API的应用程序接口(Application Program Interface)，程序员利用它们取得OS服务，该接口与实际的系统调用并不一一对应。用户程序通过调用对应的库函数使用系统调用，这些库函数与调用程序连接在一起，被嵌入在运行时装入内存的二进制程序中。



6.6.2 假脱机（SPOOLing）技术

1. 什么是SPOOLing系统？

- ❑ 早期还没有中断技术时，为了缓和CPU的高速性与I/O设备低速性间的矛盾，曾引入了脱机输入、脱机输出技术。
- ❑ 该技术是利用专门的**外围控制机**，将低速I/O设备上的数据传送到高速磁盘上；或者相反。





6.6.2 假脱机（SPOOLing）技术

1. 什么是SPOOLing系统？

- 通过多道程序技术，可以将一台物理CPU虚拟为多台逻辑CPU，从而允许多个用户进程共享一台主机；
- 通过假脱机技术，可将一台物理I/O设备虚拟为多台逻辑I/O设备，从而允许多个用户进程共享一台物理I/O设备。



6.6.2 假脱机（SPOOLing）技术

1. 什么是SPOOLing系统？

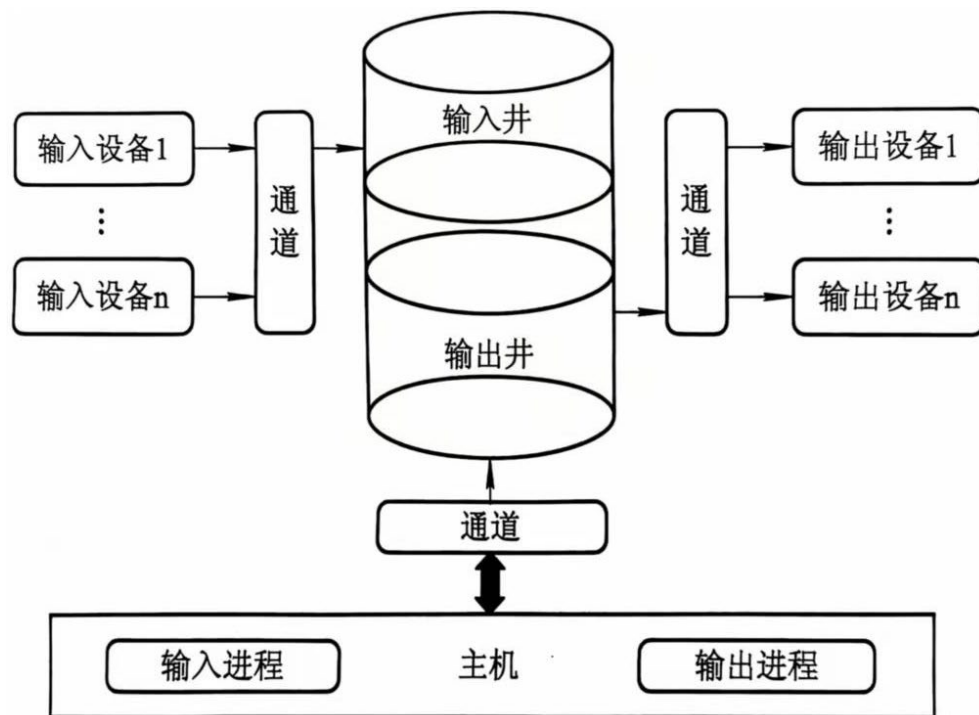
- ❑ 系统引入多道程序技术后，可以利用其中的一道程序来模拟脱机输入时的外围控制机功能，将低速输入I/O设备的数据传到高速磁盘；再用另一道程序模拟输出时的外围控制机功能，把数据从磁盘传到低速输出设备，这样，便可在主机的直接控制下，实现脱机输入/输出功能。此时外围操作与CPU对数据的处理同时进行。把这种在联机情况下实现的同时外围操作称为SPOOLing(Simultaneous Peripheral Operating On-Line)，或称为假脱机操作。



6.6.2 假脱机（SPOOLing）技术

2. SPOOLing系统的组成

□ SPOOLing技术是对脱机输入、输出系统的模拟，建立在具通道技术和多道程序技术的基础上，以高速随机外存（通常为磁盘）为后援存储器。





6.6.2 假脱机（SPOOLing）技术

2. SPOOLing系统的组成

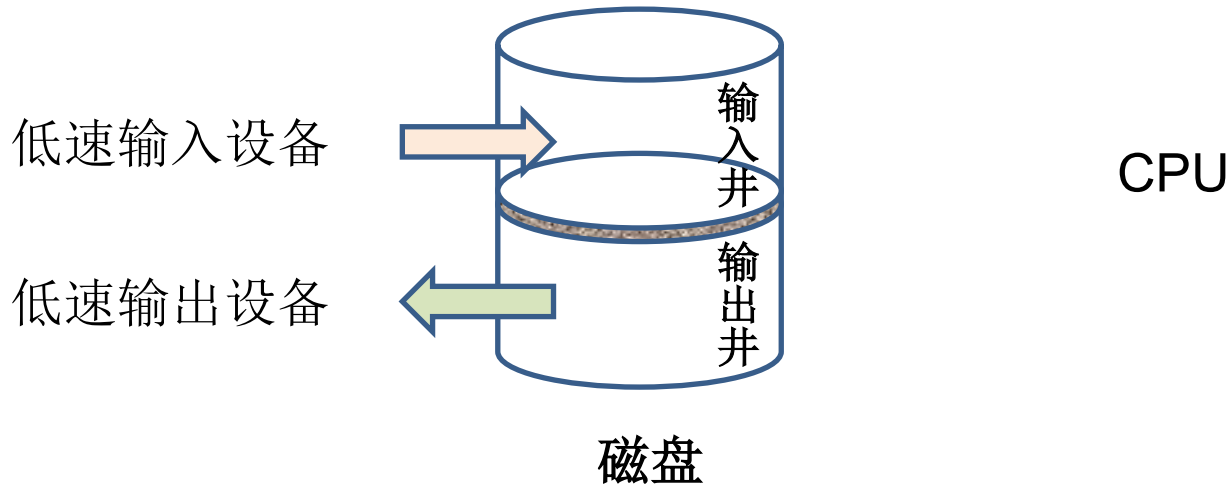
□ SPOOLing系统主要有以下四部分组成：

- (1) 输入井和输出井
- (2) 输入缓冲区和输出缓冲区
- (3) 输入进程 SP_i 和输出进程 SP_o 。
- (4) 井管理程序



(1) 输入井和输出井

- **输入井和输出井**是在**外存(磁盘)**上开辟的两个存储空间。其数据一般以**文件**的形式组织管理。
- **输入井**：模拟脱机输入时的磁盘设备，用于暂存低速I/O设备输入的数据。
- **输出井**：模拟脱机输出时的磁盘，用于暂存用户程序的输出数据。





(2) 输入缓冲区和输出缓冲区

- ❑ 输入缓冲区和输出缓冲区是在**内存**中开辟的两个缓冲区。用于缓和CPU和磁盘之间速度不匹配的矛盾。
- ❑ **输入缓冲区**：用于暂存由输入设备送来的数据，以后再送入输入井。
- ❑ **输出缓冲区**：用于暂存从输出井送来的数据，以后再传送给输出设备。



（3）输入进程 SP_i 和输出进程 SP_o 。

- ❑ 输入进程和输出进程是在**内存中的两个进程**。
- ❑ **输入进程**：又叫预输入进程，用于模拟脱机输入时的外围控制机，将用户要求的数据从输入设备通过输入缓冲区再送到输入井。当CPU需要输入数据时，直接从输入井读入内存。
- ❑ **输出进程**：又叫缓输出进程，用于模拟脱机输出时的外围控制机，把用户要求输出的数据，先从内存送到输出井，待输出设备空闲时，再将输出井中的数据经过输出缓冲区送到输出设备。



(4) 井管理程序

- 井管理程序：用于**控制作业和磁盘井之间信息的交换**。当作业执行过程中，向某台设备发出启动输入和输出请求时，由操作系统调用井管理程序，由其控制着从输入井读取信息或将从输出井输出信息。



3. SPOOLing系统的特点

□ 提高了I/O的速度。

从低速外设进行的I/O操作，演变成对输入井输出井数据的存取，如同脱机输入输出一样，提高I/O速度。

□ 将独占设备改造为共享设备。

没有为任何进程分配设备，只是在输入井或输出井中为进程分配一个存储区和建立一张I/O请求表。

□ 实现了虚拟设备功能。

宏观上，多个进程同时使用一台设备，对每个进程而言，又好像自己独占一样。实现了从独占到共享的虚拟功能。



4. 例子：共享打印机

- ❑ 打印机属于**独占设备**。利用假脱机技术可将它改造为一台可供多个用户共享的设备，从而提高设备的利用率，方便用户。
- ❑ 共享打印机技术已被广泛地用于多用户系统和局域网络中。
- ❑ 假脱机打印系统主要有以下三部分：
 - (1) 磁盘缓冲区：位于磁盘。用于存放所有用户的打印输出数据，属于输出井。
 - (2) 打印缓冲区：位于内存。用于暂存输出井送来的数据，以后再传送给打印机。
 - (3) 假脱机管理进程和假脱机打印进程。



4. 例子：共享打印机

□ 当用户进程提出打印请求时，SPOOLing系统**立即同意**其打印请求，但并不真正把打印机分配给它们，而是由假脱机管理进程为每个进程做两件事情。

① 在磁盘缓冲区中为它申请一个**空闲磁盘块区（输出井）**，并将用户进程要打印的数据从内存送入磁盘缓冲区。

② 再为用户进程申请一张空白的**用户打印请求表**，并将用户的打印请求填入表中（用于说明打印数据在磁盘缓冲区的位置），将该表挂到**打印队列上**。

至此，用户进程觉得它的打印过程已经完成，而不必等待真正慢速的打印过程的完成。



4. 例子：共享打印机

- ❑ 当**打印机空闲**时，假脱机打印进程将从请求队列队首取出一张打印申请表，根据表中的要求将打印的数据从输出井传送到内存缓冲区，再由打印机进行输出打印。打印完后，处理队列中的下一个打印请求。
- ❑ 这样，虽然系统中只有一台打印机，但系统并未将它分配给任何进程，而只是为每个提出打印请求的进程在输出井中分配一个存储区（相当于一个逻辑设备），使每个用户进程都觉得自己在独占一台打印机，从而实现了打印机的共享。



5. 守护进程

假脱机系统来实现打印机共享的方案可以做某些修改，如**取消该方案中的假脱机管理进程**，为打印机**建立一个守护进程**，以及一个特殊目录，称为**假脱机目录**。为了打印一个文件，一个进程首先要生成需要打印的整个文件并把它放在假脱机目录里。由守护进程打印该目录下的文件，该守护进程是允许使用打印机设备文件的唯一进程。

凡是需要将独占设备改造为多个进程共享的设备时，都需要配置一个守护进程和一个假脱机目录。



6.7 缓冲区管理

- 6.7.1 缓冲区的引入
- 6.7.2 单缓冲区与双缓冲区
- 6.7.3 环形缓冲区
- 6.7.4 缓冲池（**Buffer Pool**）



6.7 缓冲区管理

□ 现代OS中，几乎所有I/O设备与CPU交换数据时都使用缓冲区。

□ 缓冲区特点：

- 当缓冲区**非空**，不能往缓冲区写入数据，只能从缓冲区取数据；
- 当缓冲区为**空**，可以往缓冲区写入数据，但必须充满后，才能从缓冲区把数据传出。

□ 缓冲的实现：

- **采用专用的硬件缓冲区**。如设备控制器中的数据缓冲寄存器。成本高，一般配备的数量少。
- **在内存中划出一块专门的空间**，作为缓冲区来存放输入输出的数据，又称“**软缓冲**”。



6.7.1 缓冲区的引入

□ 为什么引入缓冲区？

- (1) 缓和CPU与I/O设备间速度不匹配的矛盾。
- (2) 减少对CPU的中断频率，放宽对CPU中断响应时间限制。
- (3) 解决数据粒度不匹配的问题。
- (4) 提高CPU和I/O设备之间的并行性。

例如，若没有缓冲区，输出数据时，由于打印机的速度跟不上而使CPU停下来等待。设置缓冲区，可以暂存需打印的数据，由打印机慢慢从缓冲区中取出数据打印，CPU 可以去做其他工作。

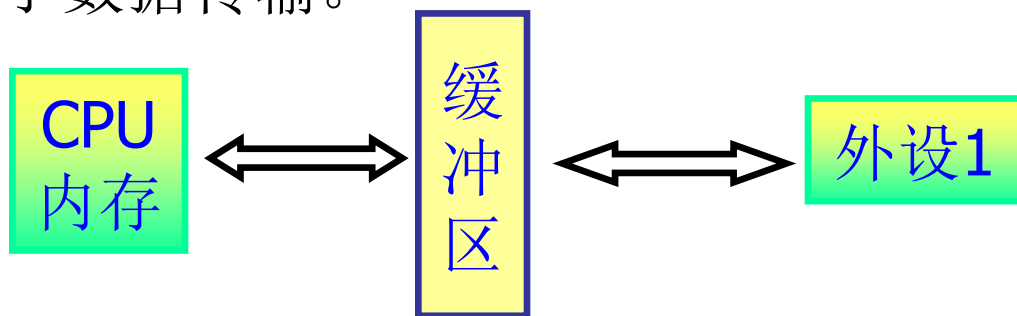
- ### □ 根据缓冲区的数目和使用方式的不同，分为单缓冲、双缓冲、循环缓冲、缓冲池等。



6.7.2 单缓冲区与双缓冲区

1. 单缓冲

- 当用户进程发出I/O请求时，OS在内存系统区为之分配一个缓冲区用于数据传输。



- 块设备输入时，假设从磁盘传送数据块到缓冲区的时间为 T ，OS将缓冲区数据传到用户区的时间为 M ，CPU对此数据块的处理时间为 C 。则 T 和 C 是可以并行的，可以提高I/O的速度。单缓冲区系统对每一块数据的处理时间为： $\text{Max}(C, T) + M$ 。

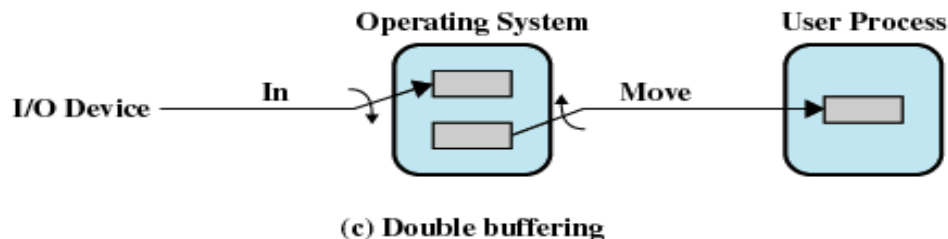
注意：CPU在传送时间 M 内处于空闲。



6.7.2 单缓冲区与双缓冲区

2. 双缓冲

- ❑ 为了加快I/O速度和实现双向传输，引入了双缓冲，又称缓冲对换（Buffer Swapping）。
- ❑ I/O设备输入数据时先装入缓冲区1，1满后再装入缓冲区2；同时，CPU可先从缓冲区1取数据给用户进程处理，完成后，若缓冲区2满，继续从2取数据放入用户进程。
- ❑ 块设备输入时，双缓冲区系统对每一块数据的处理时间约为： $\text{Max}(C, T)$

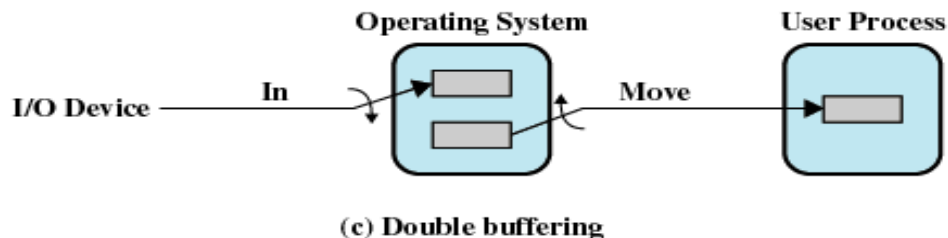




6.7.2 单缓冲区与双缓冲区

2. 双缓冲

- ❑ 可以实现**双向传输**，一个做为输入缓冲区用，一个做为输出缓冲区用。
- ❑ 输出字符时，若上次输出还在缓冲区没有取走，可以输出到第二个缓冲区，从而避免了CPU阻塞，提高系统效率。





6.7.3 环形缓冲区

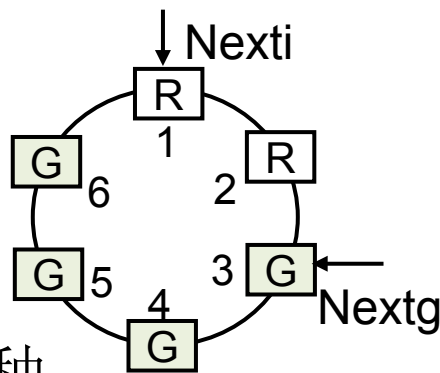
- ❑ 如果需要进行大量的输入输出操作，仅有两个缓冲区还是不够的。考虑使用**多缓冲区机制**。
- ❑ 如前面介绍的**生产者-消费者模型中的有界缓冲区**就是使用多缓冲区机制，即作为输入用，又可以作为输出用。



1. 环形缓冲区的组成

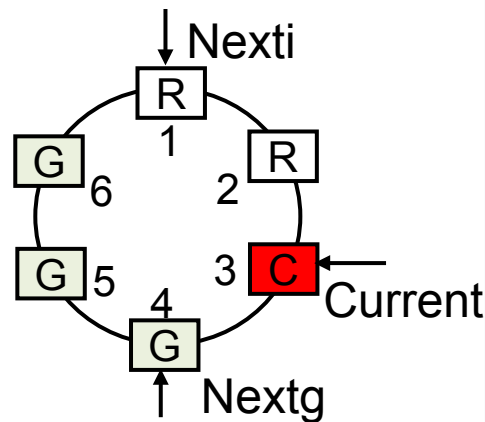
(1) 多个大小相等的缓冲区。可分3类：

- 空缓冲区R
- 满缓冲区G
- 现行缓冲区C：正在使用的



(2) 多个指针。输入缓冲区有3种：

- 指针Nextg：指示计算进程下一个可用缓冲区G
- 指针Nexti：指示输入进程下次可用缓冲区R
- 指针Current：指示计算进程正在使用的缓冲区C





2. 环形缓冲区的使用

计算进程和输入进程利用下述两个过程使用环形缓冲区：

(1) Getbuf过程

- **计算进程**使用缓冲区时，调用Getbuf过程取数据：将Nextg所指缓冲区改为“现行”，用Current指向它，同时将Nextg指向下一个G缓冲区。
- **输入进程**使用缓冲区时，调用Getbuf过程写数据：将Nexti所指缓冲区给输入进程使用，同时将Nexti指向下一个R缓冲区。

(2) Releasebuf过程

- **计算进程**把C缓冲区数据提取完毕时，调用Releasebuf过程，将缓冲区G释放，将该缓冲区由现行缓冲区C改为空缓冲区R。
- **输入进程**把缓冲区装满时，调用Releasebuf过程，将该缓冲区释放，并改为满缓冲区G。



3. 进程之间的同步问题

使用输入循环缓冲，可使输入进程和计算进程并行执行。相应地，指针**Nexti**和指针**Nextg**不断沿顺时针方向移动，可能会有以下两种情况：

（1）**Nexti**指针追上**Nextg**指针。已无空缓冲区，输入进程阻塞——计算进程在**Releasebuf**时唤醒它。

（2）**Nextg**指针追上**Nexti**指针。计算进程快，已无满缓冲区。计算进程阻塞，直到输入进程用**Releasebuf**时唤醒它。



6.7.4 缓冲池（Buffer Pool）

- ❑ 上述缓冲区仅适用于特定的I/O进程和计算进程，因而它属于**专用缓冲**。
- ❑ 当系统较大时，将会有许多这样的循环缓冲，这不仅要消耗大量的内存空间，而且利用率不高。为了**提高缓冲区利用率**，**目前广泛流行缓冲池**，在池中设置了多个可供若干进程共享的缓冲区。
- ❑ 缓冲区和缓冲池的区别：
 - 缓冲区是一组内存块的链表；
 - 缓冲池包含一个管理的数据结构和一组操作函数的管理机制，用于管理多个缓冲区。



6.7.4 缓冲池（Buffer Pool）

1. 缓冲池的组成

缓冲池管理多个缓冲区，**每个缓冲区由用于标识和管理的缓冲首部、用于存放数据的缓冲体两部分组成**。一般将缓冲池中具有相同类型的缓冲区链接成一个队列，形成以下三个队列：

- 空缓冲队列**emq**：头指针 $F(emq)$ ，尾指针 $L(emq)$
- 输入队列**inq**：头指针 $F(inq)$ ，尾指针 $L(inq)$
- 输出队列**outq**：头指针 $F(outq)$ ，尾指针 $L(outq)$



6.7.4 缓冲池 (Buffer Pool)

2. Getbuf和Putbuf过程

设type指向队列，互斥信号量MS(type)，资源信号量RS(type)

Procedure Getbuf(type) //从type所指队列的队首摘取一个缓冲区

```
{ wait(RS(type));
```

```
wait(MS(type));
```

```
B(number)=Takebuf(type);//从type所指队列的队首摘取一个缓冲区
```

```
signal(MS(type)); }
```

Procedure Putbuf(type, number)

```
{ wait(MS(type)) ;
```

```
Addbuf(type,number);//将number所指缓冲区挂到type所指队列上
```

```
signal(MS(type)) ;
```

```
signal(RS(type)) ; }
```

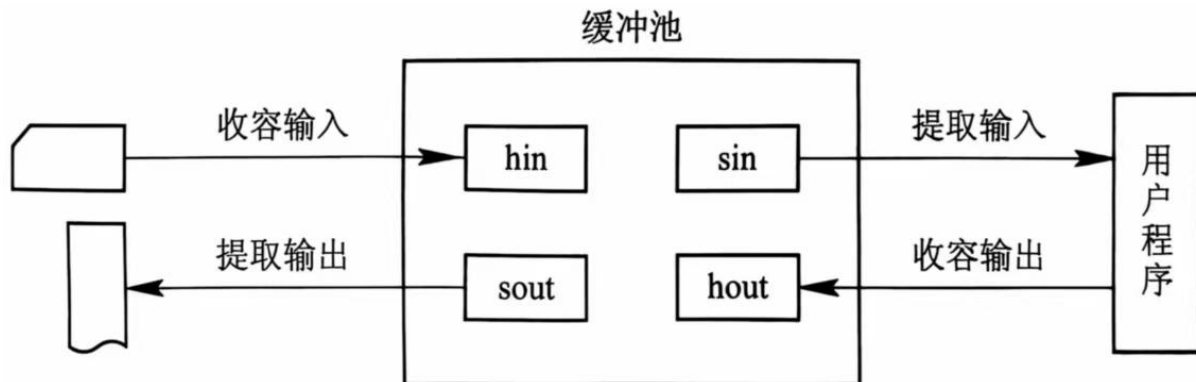


6.7.4 缓冲池 (Buffer Pool)

3. 缓冲池的工作方式

(1) 收容输入

输入进程调用**Getbuf(emq)**过程，从空缓冲队列**emq**队首摘下一个空缓冲区，作为收容输入工作缓冲区**hin**。然后将数据输入其中，装满后再调用**Putbuf(inq, hin)**过程，将该缓冲区挂到输入队列**inq**上。



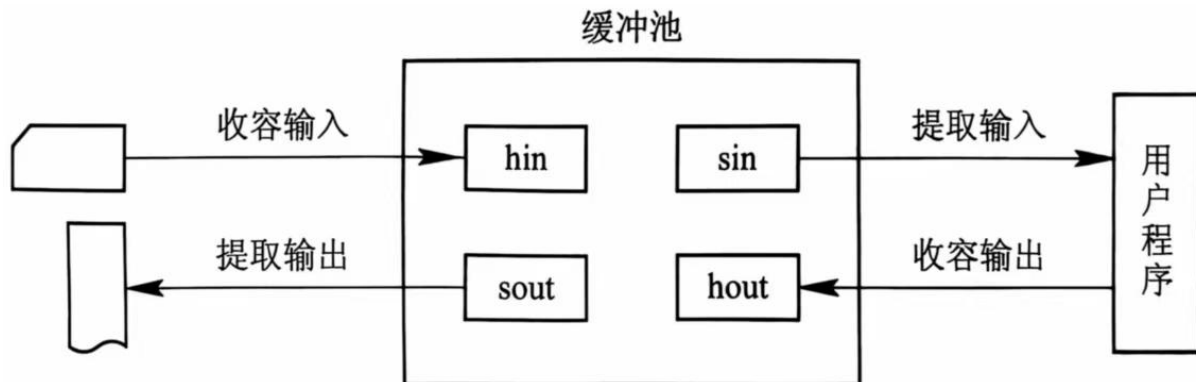


6.7.4 缓冲池 (Buffer Pool)

3. 缓冲池的工作方式

(2) 提取输入

计算进程调用**Getbuf(inq)**过程，从输入缓冲队列**emq**的队首摘下一个缓冲区，作为提取输入工作缓冲区**sin**，计算进程从中提取数据。计算进程用完数据后，再调用**Putbuf(emq, sin)**过程，将该缓冲区挂到空缓冲队列**emq**上。



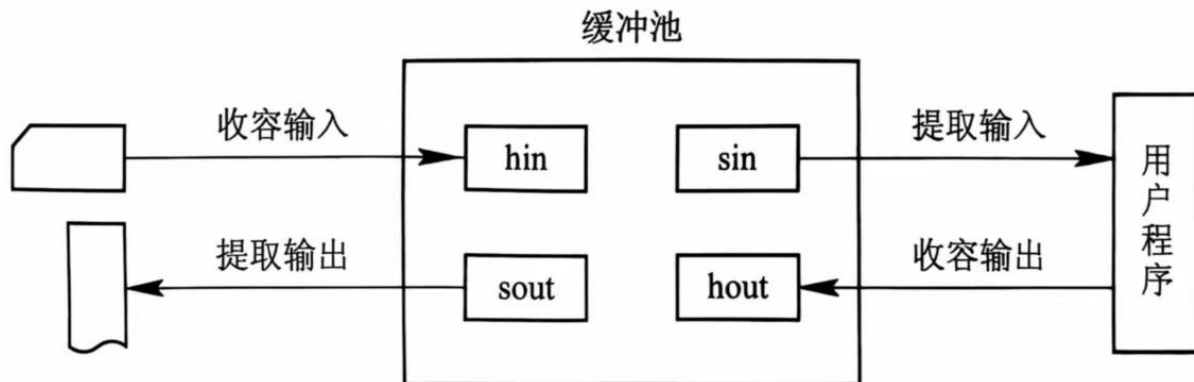


6.7.4 缓冲池 (Buffer Pool)

3. 缓冲池的工作方式

(3) 收容输出

计算进程调用Getbuf(emq)过程，从空缓冲队列emq的队首摘下一个空缓冲区，把它作为收容输出工作缓冲区hout。当其中装满输出数据后，调用Putbuf(outq, hout)过程，将该缓冲区挂到输出队列outq上。



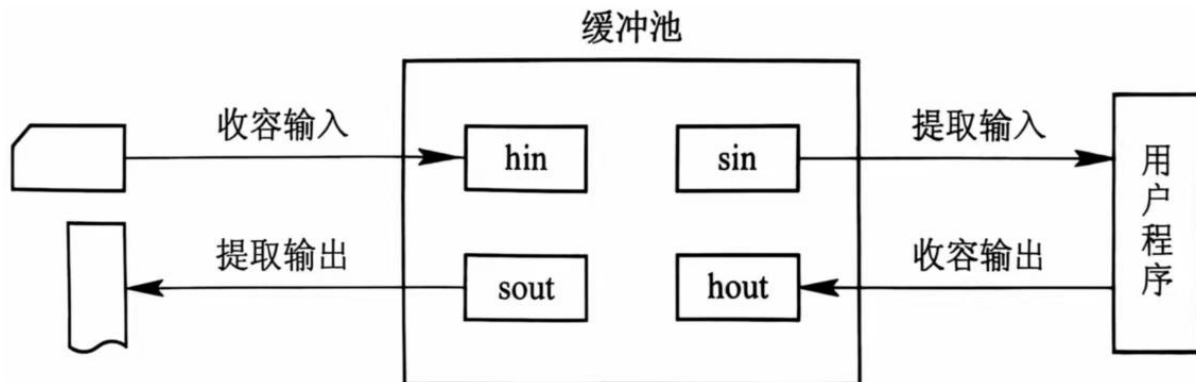


6.7.4 缓冲池 (Buffer Pool)

3. 缓冲池的工作方式

(4) 提取输出

输出进程调用Getbuf(outq)过程，从输出缓冲队列outq的队首摘下一个缓冲区，把它作为提取输出工作缓冲区sout。当数据提取完后，再调用Putbuf(emq, sout)过程，将该缓冲区挂到emq队列末尾。





6.8 磁盘存储器的性能和调度

- 6.8.1 磁盘性能简述

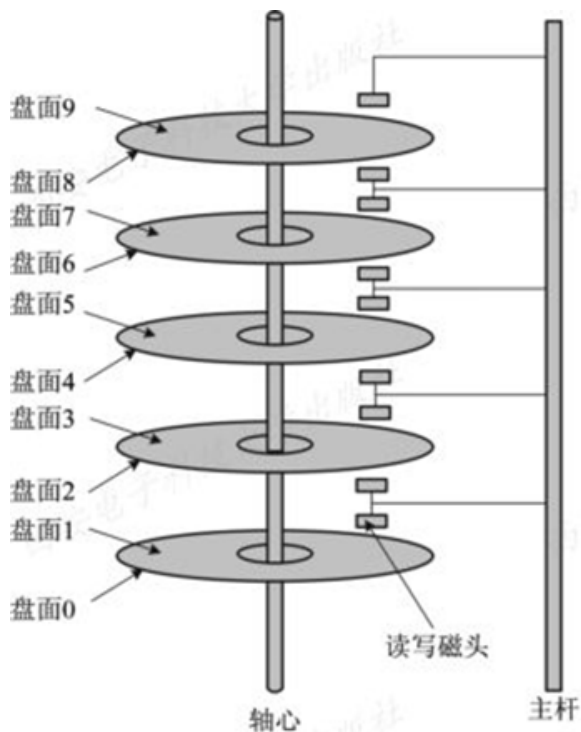
- 6.8.2 早期的磁盘调度算法

- 6.8.3 基于扫描的磁盘调度算法



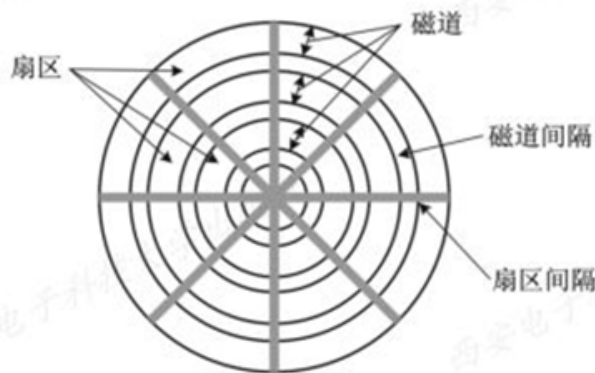
6.8.1 磁盘性能简述

1. 数据的组织和格式



(a) 磁盘驱动器的结构

- 盘面上的数据被组织成一组同心圆，称为**磁道**。
- 所有盘片上相对位置相同的磁道的组合称为**柱面**。



(b) 磁盘的数据布局

- 每个磁道有几百个**扇区（磁盘块）**，各扇区存放的数据量相同。

磁盘地址：柱面号·盘面号·扇区号（块号）



6.8.1 磁盘性能简述

2. 磁盘的类型

最常见的分类有：将磁盘分成硬盘和软盘、单片盘和多片盘、固定头磁盘和活动头(移动头)磁盘等。

(1) **固定头磁盘**：每条磁道上都有一个读写磁头，所有的磁头装在一个刚性磁臂中。主要用于大容量磁盘。

(2) **移动头磁盘**：每个盘面仅配有一个磁头，也被装入磁臂中。该磁头必须能移动以进行寻道。以串行方式读写，I/O速度较慢，结构简单，广泛应用于中小型磁盘设备中。



6.8.1 磁盘性能简述

3. 磁盘的访问时间

(1) 寻道时间 T_s : 把磁臂（磁头）移动到指定磁道上所经历的时间。

$$T_s = m \times n + s$$

其中， m : 移动一条磁道花费的时间，常数，与磁盘驱动器速度有关；

n : 磁道数量；

s : 磁臂启动时间。



6.8.1 磁盘性能简述

3. 磁盘的访问时间

(2) 旋转延迟时间 T_r : 指定扇区移动到磁头下面所经历的时间，和磁盘旋转速度有关。

$$T_r = \frac{1}{2r}$$

其中， r : 磁盘每秒的转数

旋转延迟时间与磁盘旋转速度线性相关。



6.8.1 磁盘性能简述

3. 磁盘的访问时间

(3) 传输时间 T_t : 把数据从磁盘读出或向磁盘写入数据所经历的时间。与每次读写的字节数 b 和旋转速度有关。

$$T_t = \frac{b}{Nr}$$

其中， r : 磁盘每秒的转数

N : 一条磁道上的字节数

传输时间与磁盘旋转速度线性相关。



6.8.1 磁盘性能简述

3. 磁盘的访问时间

(4) 总平均存取时间 T_a ：

$$\begin{aligned} T_a &= T_s + T_\tau + T_t \\ &= m \times n + s + \frac{1}{2r} + \frac{b}{Nr} \end{aligned}$$

- ❑ 在磁盘的平均存取时间中，寻道时间 T_s 占大部分，它与磁盘直径以及磁盘的调度算法密切相关。
- ❑ 从调度算法层面降低寻道时间，应该尽量把信息放到同一个或相邻的柱面上。



6.8.2 早期的磁盘调度算法

- ❑ 磁盘调度算法的目标：使磁盘的平均寻道时间最少。
- ❑ 目前常用的磁盘调度算法：先来先服务算法、最短寻道时间优先算法、扫描算法。

适用于请求磁盘的进程数目较少的场合



1. 先来先服务 (FCFS) 简单、公平。

□ 思想：根据进程请求访问磁盘的先后次序进行调度。

【例】若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，设磁头初始处于100柱面，则对于FCFS算法，平均寻道长度为多少？

总寻道长度=

$45+3+19+21+72+70+10+112+146=498$

平均寻道长度= $498/9=55.3$

(从100号磁道开始)	
被访问的 下一个磁道号	移动距离 (磁道数)
55	45
58	3
39	19
18	21
90	72
160	70
150	10
38	112
184	146
平均寻道长度：55.3	

若干个等待访问磁盘者依次要访问的柱面为**20, 44, 40, 4, 80, 12, 76**，假设每移动一个柱面需要**3**毫秒时间，移动臂当前位于**40**号柱面，请按下列算法分别计算为完成上述各次访问总共花费的寻道时间。

(1) 先来先服务算法;

从 40 号磁道开始	
下一磁道	移动磁道数
20	20
44	24
40	4
4	36
80	76
12	68
76	64
总寻道长度: 292	

总寻道时间=**292**×**3ms**





2. 最短寻道时间优先SSTF

公平性差，
有“饿死”现象。

□ 思想：优先调度与当前磁头所在磁道距离最近的磁道。

【例】若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，设磁头初始处于100柱面，则对于SSTF算法，平均寻道长度为多少？

服务顺序：90,58,55,39,38,18,150,160,184

总寻道长度

$=10+32+3+16+1+20+132+10+24=248$

平均寻道长度 $=248/9=27.56$

(从100号磁道开始)

被访问的 下一个磁道号	移动距离 (磁道数)
90	10
58	32
55	3
39	16
38	1
18	20
150	132
160	10
184	24
平均寻道长度：27.5	

若干个等待访问磁盘者依次要访问的柱面为**20, 44, 40, 4, 80, 12, 76**, 假设每移动一个柱面需要**3**毫秒时间, 移动臂当前位于**40**号柱面, 请按下列算法分别计算为完成上述各次访问总共花费的寻道时间。

(2) 最短寻找时间优先算法。

从 40 号磁道开始	
下一磁道	移动磁道数
40	0
44	4
20	24
12	8
4	8
76	72
80	4
总寻道长度: 120	

$$\text{总寻道时间} = 120 \times 3\text{ms}$$



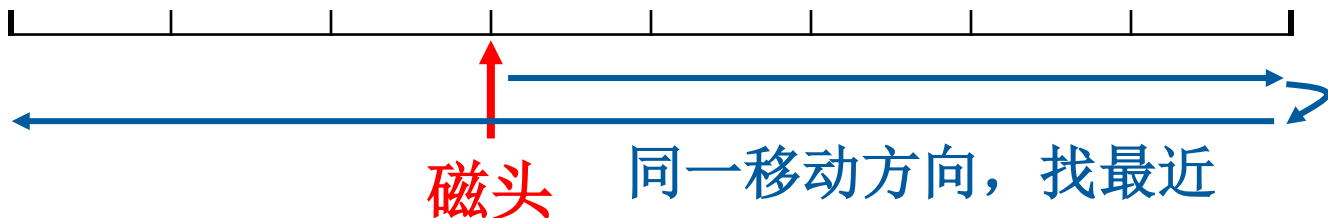


6.8.3 基于扫描的磁盘调度算法

1. 扫描（SCAN）算法——电梯算法

□ **思想：** 不仅考虑欲访问磁道与当前磁头所在磁道距离，更优先考虑磁头当前的移动方向。即**优先调度磁头同一移动方向上距离最近的**，直至同一方向上的磁道访问完毕，再切换磁头移动方向。

既获得了较好性能，又能防止“饥饿”现象。





1. 扫描（SCAN）算法——电梯算法

【例】若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，设磁头初始处于100柱面，则对于SCAN算法，平均寻道长度为多少？设磁头正向磁道号增加方向移动。

解：

服务顺序：150,160,184,90,58,55,39,38,18

总寻道长度

$$=50+10+24+94+32+3+16+1+20=250$$

$$\text{平均寻道长度}=250/9=27.78$$

(从100# 磁道开始，向磁道号
增加方向访问)

被访问的 下一个磁道号	移动距离 (磁道数)
150	50
160	10
184	24
90	94
58	32
55	3
39	16
38	1
18	20
平均寻道长度：27.8	

若干个等待访问磁盘者依次要访问的柱面为**20, 44, 40, 4, 80, 12, 76**, 假设每移动一个柱面需要3毫秒时间, 移动臂当前位于**40**号柱面, 请按下列算法分别计算为完成上述各次访问总共花费的寻道时间。

(3) 电梯算法。

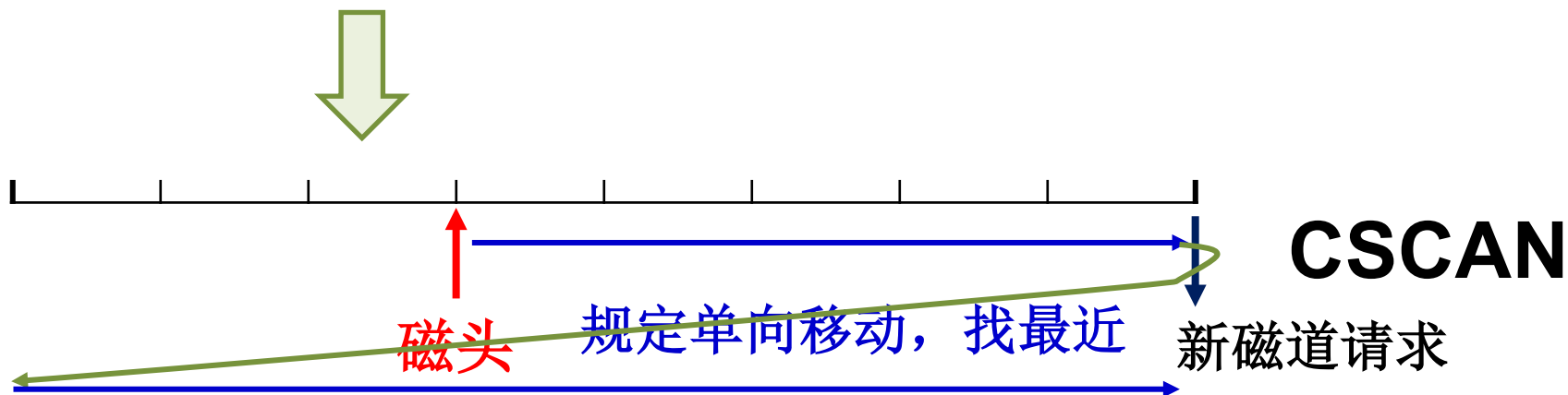
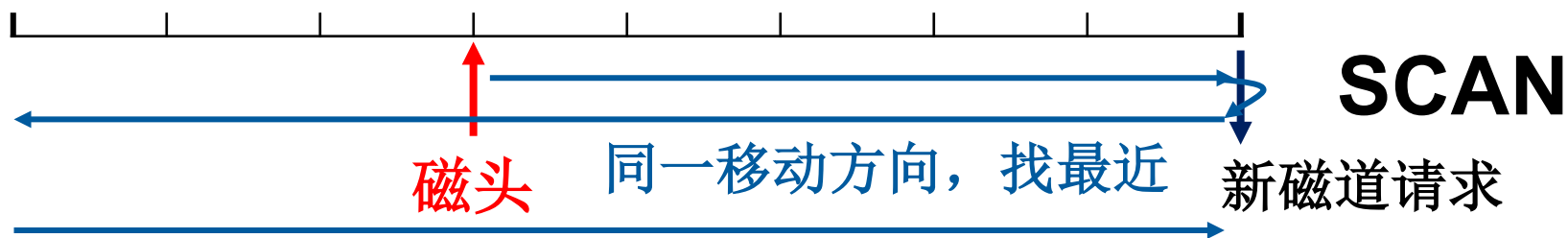
从 40 号磁道开始, 先向右	
下一磁道	移动磁道数
40	0
44	4
76	32
80	4
20	60
12	8
4	8
总寻道长度: 116	

从 40 号磁道开始, 先向左	
下一磁道	移动磁道数
40	0
20	20
12	8
4	8
44	40
76	32
80	4
总寻道长度: 112	





1. 扫描（SCAN）算法——电梯算法





6.8.3 基于扫描的磁盘调度算法

2. 循环扫描（CSCAN）算法——单向扫描

□ **思想：** 规定磁头单向移动，当磁头访问完同一方向最后一个磁道后，磁头立即返回反方向最远磁道，然后沿同方向扫描。

【例】若递交给磁盘驱动程序的磁盘柱面请求按到达时间顺序分别是55、58、39、18、90、160、150、38、184，设磁头初始处于100柱面，则对于FCFS算法，平均寻道长度为多少？设磁头向磁道号增加方向移动。

解： 服务顺序：150,160,184,18,38,39,55,58,90

总寻道长度=50+10+24+166+20+1+16+3+32=322

平均寻道长度=322/9=35.78

(从100# 磁道开始，向磁道号增加方向访问)

被访问的下一个磁道号	移动距离(磁道数)
150	50
160	10
184	24
18	166
38	20
39	1
55	16
58	3
90	32
平均寻道长度：35.8	

若干个等待访问磁盘者依次要访问的柱面为**20, 44, 40, 4, 80, 12, 76**, 假设每移动一个柱面需要3毫秒时间, 移动臂当前位于**40**号柱面, 请按下列算法分别计算为完成上述各次访问总共花费的寻道时间。

(4) 循环扫描算法。

从 40 号磁道开始, 先向右	
下一磁道	移动磁道数
40	0
44	4
76	32
80	4
4	76
12	8
20	8
总寻道长度: 132	

从 40 号磁道开始, 先向左	
下一磁道	移动磁道数
40	0
20	20
12	8
4	8
80	76
76	4
44	32
总寻道长度: 148	





河南大學
HENAN UNIVERSITY

明德，新民
止于至善



The End ...

